UNIVERSITÀ DI PISA

DEPARTMENT OF COMPUTER SCIENCE

Master's degree in Computer Science

THESIS

Local completeness logic on strategic term rewriting

Student:
Niccolò Piazzesi

Supervisor:
Roberto Bruni

ACADEMIC YEAR 2021/2022

Abstract

In Abstract Interpretation, completeness is an ideal situation in which the abstraction does not introduce any false alerts. Completeness is notoriously difficult to achieve in practice, because it is a *global* property: it must hold for any possible input. The weakened notion of *local completeness* relaxes this requirement by considering only some specific inputs, finding wider applicability in practice. **Local Completeness Logic** (LCL) is a novel proof system that can reason about both correctness and incorrectness of locally complete computations, by combining over and under-approximating static analysis. In this thesis, we study the applicability of the LCL framework to the context of **strategic rewriting**, where the classical notion of term rewriting is enriched by the definition of **strategies**, whose purpose is to guide and control the application of rules. We present a new strategy language, **Strat**, and extend the LCL proof system to reason about properties of Strat programs.

Contents

1 Introduction **1**
- 1.1 Local Completeness Logic . . . 2
- 1.2 Strategic term rewriting . . . 2
- 1.3 Contributions . . . 3
- 1.4 Outline . . . 4

2 Background **6**
- 2.1 Abstract interpretation . . . 6
  - 2.1.1 Galois connections . . . 8
  - 2.1.2 Abstract Semantics . . . 11
- 2.2 Term rewriting . . . 12
  - 2.2.1 Abstract Reduction Systems . . . 14
  - 2.2.2 Signatures and universal algebra . . . 15
  - 2.2.3 Term rewriting systems . . . 18
  - 2.2.4 Strategic rewriting . . . 21

3 Strat language **25**
- 3.1 The language . . . 25
- 3.2 Semantics . . . 26
  - 3.2.1 Expressions . . . 27
  - 3.2.2 Commands . . . 31

4 Local Completeness Logic on a Strategy Language **38**
- 4.1 Local completeness Logic . . . 38
- 4.2 LCL on Strat . . . 42
  - 4.2.1 Program Properties for Strat . . . 42
  - 4.2.2 Adapting proofs . . . 43
  - 4.2.3 Procedure calls . . . 44
- 4.3 Soundness . . . 45
  - 4.3.1 Soundness of the abstract semantics . . . 46
  - 4.3.2 Logical soundness . . . 51

4.4 On logical completeness 55

5 Analysis **57**
5.1 Term shape 58
5.2 Even Numbers 62

6 Conclusions **66**
6.1 Related work 66
6.2 Future work 67

Appendices **74**

A Partially Ordered Sets **74**
A.1 Fixpoints 79

List of Figures

2.1 The galois connection relation. 9
2.2 A Galois insertion. 10
2.3 Constraints for a Galois connection. 10
2.4 Church-Rosser property, confluence, and semi-confluence. . 15
2.5 Tree representation of the term $t = s(add(x, s(s(0))))$. . . 17
2.6 Rewriting of a sub-term at position p, using the equivalency (l, r) and the substitution σ. 19

3.1 Syntax of the strategy language. 26
3.2 Semantics of expressions. 28
3.3 Semantics of operators. 29
3.4 Match function. 30
3.5 Congruence function. 31
3.6 Semantics of commands. 32

4.1 Concrete semantics of regular commands. 41
4.2 Abstract semantics of regular commands. 41
4.3 The proof system LCL_A. 41
4.4 Abstract semantics for Strat (minus strategy calls). 45

5.1 Abstract domain for the first example. 59
5.2 Derivation of (5.1). 61
5.3 Derivation of (5.2). 61
5.4 Abstraction for the even program. 63
5.5 Derivation of $\vdash_A [P = \{even(n) \mid n \in Even\}]\ c\ [\{true\}]$. . 64
5.6 Derivation of $\vdash_A [P']\ c\ [\{true, false\}]$. 64

A.1 The Hasse diagram for $(\mathcal{P}(\{a, b, c\}), \subseteq)$. 75
A.2 A visualization of the difference between **maximum** and **maximal** element. S_1 has greatest element e, while S_2 has a, b, and c as maximal elements but no maximum (because they are not comparable). 76

A.3 Two subsets of a poset and their upper bounds. 77
A.4 A non-monotone function. 78
A.5 Functions on posets (red and blue arrows). (a) is monotone and has two fixpoints, namely a and $\top$. (b) is not monotone and has no fixpoint. 80
A.6 Knaster-Tarski theorem. 81
A.7 Kleene fixpoint theorem. 82

Chapter 1

Introduction

Reliable technology is a crucial need in the modern age. As we move in an increasingly digital world, software bugs and failures can have dire consequences, both economically and socially. Software testing and rigorous code standards are best practices that the majority of software developers adopt nowadays, to avoid such consequences. However, in many industries, "best practices" are simply not enough. Stronger guarantees are needed, to ensure that unexpected failures cannot happen. **Formal methods** employ mathematical reasoning to provide such guarantees. Among the many models provided in the field, **static analysis** has proven to be one of the major methodologies to verify correctness of software [NNH04, DFLO19, CDD$^+$15].

In static analysis, symbolic reasoning techniques are adopted to prove properties about programs without actually running them. Given a program c and a property specification $Spec$, static analysis either proves that c satisfies $Spec$ or it raises some alerts suggesting which behaviors *may* violate $Spec$. The conditional is required, since all verifier approximates the actual behavior: this is unavoidable, as proving every non-trivial property of a program is in general *undecidable* (see Rice's Theorem [HMU07]). Traditionally, the approach used is *conservative*: the actual semantics are over-approximated. When the analysis proves that $Spec$ is satisfied, we are sure that $Spec$ is true for all concrete behaviors (**soundness**). On the other hand, we may introduce *false alerts*: actions that are not present in the original program and that violates $Spec$ (also called *spurious counterexamples*). The less false alerts a verifier reports, the more it is *credible*. In recent years, the opposite perspective has been explored [O'H19, MYE21, LRV$^+$22]: exploiting under-approximation, any violation exposed is a true alert (**completeness**). This is great when the target is to *find bugs*, but of course $Spec$ may still be violated even if no alerts are raised.

1.1 Local Completeness Logic

Abstract interpretation [CC77, CC79] is a popular technique to design sound over-approximations of the program semantics. Instead of tracking the concrete behavior of a program c, the idea is to define an abstraction A, and track only relevant properties by computing the abstract semantics in A instead. To ensure soundness, we must compute an over-approximation of the actual execution. This guarantees that if a property is satisfied in A, it will also hold in the concrete. The ideal situation would be to define an abstraction that is sound and **complete**, not rasing any false alert. Unfortunately, completeness is notoriously difficult to achieve in abstract interpretation. The problem is that it is a *global notion*, meaning that it must hold for **any possible input P**.

Local Completeness Logic [BGGR21] (LCL) is a recent approach to static analysis that weakens the notion of completeness. Instead of requiring completeness on all possible inputs, a locally complete abstract interpretation is complete on a smaller, fixed set of inputs. Relaxing the notion allows us to prove the absence of false alerts, even for globally incomplete abstract domains. This led to the design of a novel proof system for locally complete abstract interpretation. The statements of the logic are triples $\vdash_A [P]c[Q]$, that given the precondition P, proves the validity of Q after executing c. The novelty resides in the fact that any triple has the following properties:

- Q is an under-approximation of the concrete post-condition post[c]P.
- c is locally complete for the abstraction A, given the input P
- Q and post[c]P share the same over-approximation in A.

These properties allow us to distinguish between true and false alerts, combining for the first time over and under-approximating static analysis, in the sense that post[c] is constrained between its under-approximation Q and its over-approximation $A(Q)$.

1.2 Strategic term rewriting

Term rewriting is a general computational paradigm that models the transformation of syntactic structures (terms) by the repeated application of simplification rules (**rewriting**). Terms may be strings, trees, states, and in general any kind of structured objects.

A rule $l \to r$ rewrites a pattern l into the pattern r. Both l and r are built on the same syntax of terms, but with an additional set of variables X. A

substitution σ collects bindings from variables to terms. Given a pattern l, the instantiation $\sigma(l)$ replaces each variable x in l with the term bound to x in σ. Rules can have a name $rlabel$ and are collected in a set R. Applying a rule $l \to r$ to a term t is done in two steps:

1. Match t with l by finding a sub-term s of t such that $\sigma(l) = s$.
2. Replace s in t with $\sigma(r)$, transforming t into a new term t'.

A set of rules R generates a relation $\to_R$ on terms. A **derivation** on R is a sequence of rewriting steps $t_0 \to_R t_1 \to_R \ldots$. When the derivation arrives at a state t_n such that no rules can be applied anymore, t_n is in **normal form**.

Strategies

The standard rewriting approach is non-deterministic: at every step different rules can be applied to different sub-terms. Choosing which rules to apply first is a fundamental issue in the implementation of term rewriting systems. The wrong choice can easily lead to infinite derivations.

Rewriting strategies are a way to control the derivation process. A strategy s is described in a **strategy language**. Strategy languages, similar to standard programming languages, provide syntactic constructs to express various combinations of rule application: sequential composition, non-deterministic choice, loops, conditional branching, and so on.

A **strategy program** consists of an initial input t, and a strategy expression S, built over a set of rules R using a strategy language. The output of the program will be a transformation of t, by applying the rules according to the order specified by S.

1.3 Contributions

Strat: a store-based strategy language

The first contribution is the introduction of a new strategy language, called **Strat**. Unlike the normal interpretation, where a strategy expression transforms an input term into a final output, **Strat** semantics are **store-based**: a program access and modifies variables storing the result of rewriting computations. The terms considered are **algebraic**, that is, they are built by composing function symbols provided by an algebraic signature Σ. We give

a detailed presentation of the language features, showing how to build complex strategies by combining basic rewriting operations. Finally, we build some toy programs and show a full execution on specific inputs.

To our knowledge, this is the first time a store-based approach to strategies is explored.

Proving Strat properties using LCL

In the second part of the thesis, we explore the problem of defining the LCL framework in the context of strategic rewriting. We start with an in-depth summary of LCL structure and main features. Then, we extend the framework to work with **Strat** programs. After a brief discussion on how to define an abstract domain for **Strat**, we present the other result of the thesis: the extension of the proof system to handle the verification of **Strat** programs. The novelty resides in the addition of a new rule for procedures. **Procedures**, or named strategies, were introduced in **Strat** to recursively traverse terms. The original presentation of LCL did not include any form of function calls, so we had to extend the proof system. We present the new rule and validate the constructions by proving that the framework is still **sound**, both from the perspective of abstract interpretation and logic.

We conclude our work presenting some toy examples, demonstrating how we used the proof system to verify a target property in a **Strat** program.

1.4 Outline

The thesis is structured as follows:

- **Chapter** 2 lays out the theoretical foundations upon which the rest of the thesis is built.
- **Chapter** 3 introduces the language **Strat**, presenting its syntax and semantics.
- **Chapter** 4 presents more in detail the LCL framework, showing how it was adapted to verify **Strat** programs. We also demonstrate the correctness of our constructions.
- **Chapter** 5 shows two case studies, where we applied LCL to prove relevant properties about toy examples.

- **Chapter** 6 draws conclusions about the work presented, and proposes some future research directions.
- **Appendix** A recalls the basic notions of order theory, providing the theoretical background to understand the thesis.

Chapter 2

Background

In this chapter, we will report some background concepts and notions that are fundamental for the rest of the thesis. Appendix A exposes the basic notions of order theory, which are assumed in the following sections.

2.1 Abstract interpretation

Abstract interpretation [CC77, CC79, CC92] is a general framework for designing sound-by-construction over-approximations of programs semantics. The central concept is to simulate a program behavior, keeping track only of the relevant properties that are being checked, while *abstracting* from the concrete values. The over-approximation makes the analysis *feasible*, allowing us to prove interesting properties about a program at the cost of false alerts. To better illustrate the idea, let us see a simple example.

Example 2.1. (Sign of arithmetic expressions) Consider a basic calculator. The calculator reads expressions like $3 \times 2 + 4 \times 5$ and computes their final value. We can think of it as an *interpreter* for this very basic programming language:

$$\begin{aligned} E \ ::= & \ z \in \mathbb{Z} \\ & \mid E + E \\ & \mid E * E \end{aligned}$$

The standard interpretation of expressions is inductively defined as a function $\mathcal{E}[\![_]\!] : E \to \mathbb{Z}$ that maps expressions to their values:

$$\begin{aligned} \mathcal{E}[\![z]\!] &= z \\ \mathcal{E}[\![E_1 + E_2]\!] &= \mathcal{E}[\![E_1]\!] + \mathcal{E}[\![E_2]\!] \\ \mathcal{E}[\![E_1 * E_2]\!] &= \mathcal{E}[\![E_1]\!] * \mathcal{E}[\![E_2]\!] \end{aligned}$$

Suppose now that we only want to track the sign of an expression. In general, we cannot do that. For example, a negative plus a positive number can be both positive or negative. To handle these cases, we must use an approximated domain of abstract values **Sign** $=$ {zero, pos, neg, $\top$}, where $\top$ represents a "don't know" value.

We can now define an abstract evaluation $\mathcal{E}_{abs} : Exp \rightarrow$ **Sign** as follows:

$$\begin{aligned}
\mathcal{E}_{abs}[\![z]\!] &= \text{pos if } z > 0 \\
\mathcal{E}_{abs}[\![z]\!] &= \text{zero if } z = 0 \\
\mathcal{E}_{abs}[\![z]\!] &= \text{neg if } z < 0 \\
\mathcal{E}_{abs}[\![E_1 + E_2]\!] &= \mathcal{E}[\![E_1]\!] \oplus \mathcal{E}[\![E_2]\!] \\
\mathcal{E}_{abs}[\![E_1 * E_2]\!] &= \mathcal{E}[\![E_1]\!] \otimes \mathcal{E}[\![E_2]\!]
\end{aligned}$$

The symbols $\oplus$ and $\otimes$ represent abstract addition and multiplication, respectively. Their semantics is defined in Table 2.1.

$\oplus$	**neg**	**zero**	**pos**	$\top$
neg	neg	neg	$\top$	$\top$
zero	neg	zero	pos	$\top$
pos	$\top$	pos	pos	$\top$
$\top$	$\top$	$\top$	$\top$	$\top$

$\otimes$	**neg**	**zero**	**pos**	$\top$
neg	neg	neg	$\top$	$\top$
zero	neg	zero	pos	$\top$
pos	$\top$	pos	pos	$\top$
$\top$	$\top$	$\top$	$\top$	$\top$

Table 2.1: Abstract addition and multiplication.

Here is an example analysis of the expression $-10*(4+7)$, where parentheses are added for precedence of evaluation:

$$\begin{aligned}
&\mathcal{E}_{abs}[\![-10 * (4 + 7)]\!] = \\
&\mathcal{E}_{abs}[\![-10]\!] \otimes \mathcal{E}_{abs}[\![4 + 7]\!] = \\
&\text{neg} \otimes (\mathcal{E}_{abs}[\![4]\!] \oplus \mathcal{E}_{abs}[\![7]\!]) = \\
&\text{neg} \otimes (\text{pos} \oplus \text{pos}) = \\
&\text{neg} \otimes \text{pos} = \\
&\text{neg}
\end{aligned}$$

The key advantage of abstract interpretation is that it uses order theory and fixpoint computations to define the abstract behavior of a program in a constructive way. It provides a safe over-approximation by establishing a strong relationship between the static analysis and the formal semantics. We now see more in detail the approach, starting with the definition of **Galois connections**.

2.1.1 Galois connections

Definition 2.2. Let C and A be two posets. A pair of monotone functions $\alpha : C \to A$ and $\gamma : A \to C$ forms a **Galois connection** if:

$$\forall c \in C, a \in A. \alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_C \gamma(a)$$

The function α is also called lower adjoint, while γ is called upper adjoint[1].

Let us see why Galois connections are fundamental.

Intuitively, C (*concrete domain*) is the powerset of the domain of program states, while A (*abstract domain*) is the set of abstract properties we are interested in verifying. Following the same reasoning, α is called *abstraction function* and γ is the *concretization function.*

The partial order relation represents a sort of 'decrease' in precision: $a \sqsubseteq_A a'$ means that the property a describes fewer states than a' (it is more precise). Analogously, $c \sqsubseteq_C c'$ simply means that c is a subset of c'.

Consider a concrete state c and its abstraction $a' = \alpha(c)$. If we take an abstract value a such that $a' \sqsubseteq_A a$, then a is still a valid abstraction of c (although less precise). That is, the fact that a is a **sound** approximation of c is expressed by

$$\alpha(c) \sqsubseteq_A a \tag{1}$$

Now consider $a'' \in A$, $c' = \gamma(a'')$ and another concrete point $c'' \sqsubseteq c'$. Then a'' is also a correct approximation of the state c'', even though c'' is more precise than c'. Following a similar reasoning as before, soundness of a'' w.r.t c'' is expressed by

$$c'' \sqsubseteq_C \gamma(a'') \tag{2}$$

When (1) and (2) are equivalent ($a'' = a$ and $c'' = c$) we obtain

$$\alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_C \gamma(a)$$

This is exactly the definition of a Galois connection (Figure 2.1). In short, using Galois connections gives us the strong guarantee that the information inferred about the program is a **safe** over-approximation of its semantics.

Example 2.3. (Sign abstraction) We can now revisit Example 2.1 in terms of a Galois connection. For the expression language E, our concrete domain C is $\mathcal{P}(\mathbb{Z})$, the powerset of integers. The abstract domain A is $\text{Sign}_\perp = \{\perp$, neg,

[1]The term adjoint comes from Category Theory, where Galois connections are a special case of a concept called *adjunction.* Such a discussion is out of scope for this work, but we refer the curious reader to [Awo10] for a comprehensive introduction to the argument.

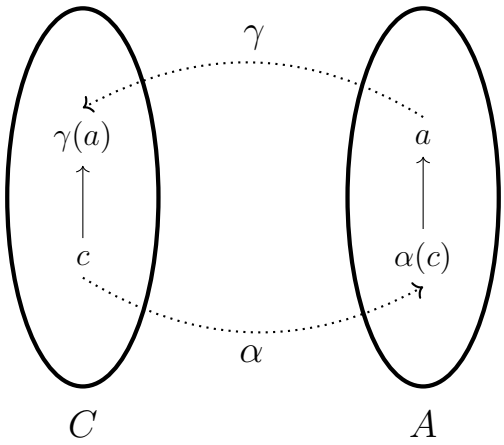


Figure 2.1: The galois connection relation.

zero, pos, $\top$ }, which is the Sign domain lifted to a complete lattice by adding a $\bot$ element (A.9). Now, we can define the abstraction and concretization function as follows:

$$\alpha : \mathcal{P}(\mathbb{Z}) \to \text{Sign} \qquad \gamma : \text{Sign} \to \mathcal{P}(\mathbb{Z})$$

$$\alpha(S) = \begin{cases} \bot & \text{if } S = \emptyset \\ \text{neg} & \text{if } \emptyset \nsubseteq S \subseteq \mathbb{Z}_{<0} \\ \text{zero} & \text{if } S = \{0\} \\ \text{pos} & \text{if } \emptyset \nsubseteq S \subseteq \mathbb{Z}_{>0} \\ \top & \text{otherwise} \end{cases} \qquad \gamma(a) = \begin{cases} \emptyset & \text{if } a = \bot \\ \{0\} & \text{if } a = \text{zero} \\ \mathbb{Z}_{>0} & \text{if } a = \text{pos} \\ \mathbb{Z}_{<0} & \text{if } a = \text{neg} \\ \mathbb{Z} & \text{otherwise} \end{cases}$$

Soundness of the approach can be proved by structural induction [Ros95].

In standard Galois connections, two abstract elements may share the same concretization. For example, imagine that in our previous sign analysis we take $\mathcal{P}(\mathbb{Z} \times \mathbb{Z})$ and $\mathcal{P}(\text{Sign} \times \text{Sign})$ as our C and A. This might be useful if we want to analyze two different expressions. Now, this abstraction is redundant. The values

$$(\bot, \text{pos}) \quad (\text{pos}, \bot) \quad (\bot, \bot)$$

get all concretized as $\emptyset \in \mathcal{P}(\mathbb{Z} \times \mathbb{Z})$.

It is possible to avoid this, by using **Galois insertions** instead.

Definition 2.4. (Galois insertion)

A Galois connection (C, A, α, γ) is called **Galois insertion** iff:

$$\alpha\gamma = id_A$$

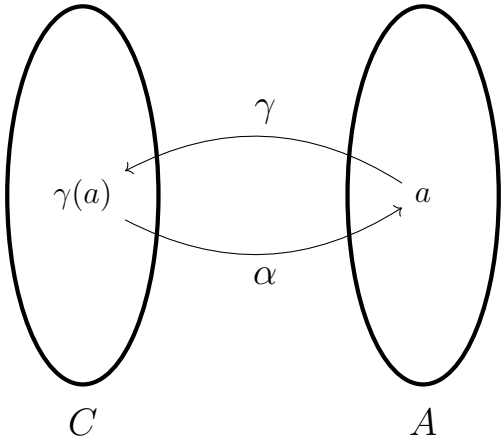


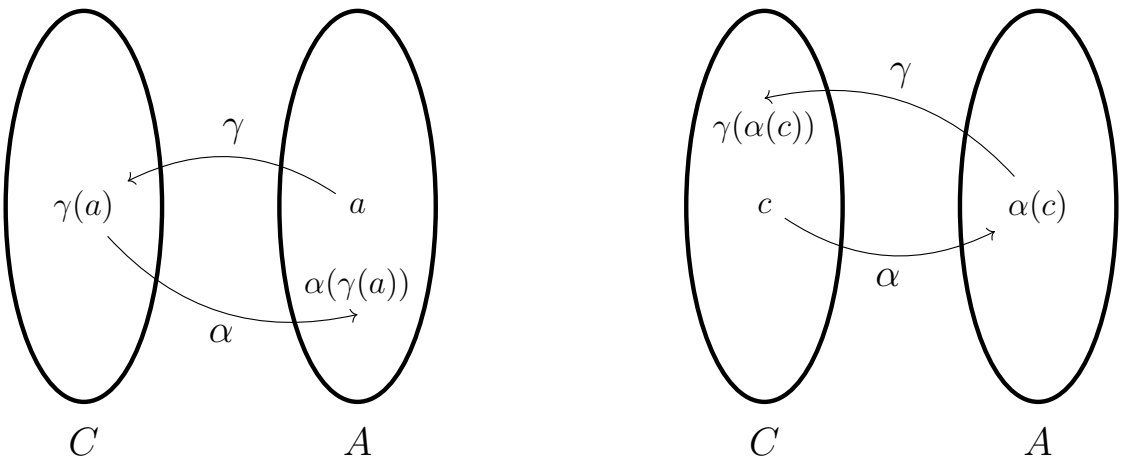
Figure 2.2: A Galois insertion.

Properties of Galois connections We recall some important properties of Galois connections and insertions.

Proposition 2.5. *Let C and A be two partially ordered sets, and $\alpha : C \to A, \gamma : A \to C$ be a pair of monotone functions between the two. (C, A, α, γ) is a Galois connection if and only if:*

- $\alpha\gamma \sqsubseteq_A id_A$
- $id_C \sqsubseteq_C \gamma\alpha$

This is an alternative but equivalent definition, and it can be shown that one implies the other. Figure 2.3 provides a visualization.



(a) $\alpha\gamma \sqsubseteq_A id_A$. (b) $id_C \sqsubseteq_C \gamma\alpha$.

Figure 2.3: Constraints for a Galois connection.

Proposition 2.6. *Let (C, A, α, γ) be a Galois connection. Then γ is co-additive and α is additive (A.13).*

In particular, $\gamma(\top_A) = \top_C$ and $\alpha(\top_C) = \top_A$.
There exists an alternative definition of Galois insertions as well.

Proposition 2.7. *Let (C, A, α, γ) be a Galois connection. The following properties are equivalent:*

- $\alpha\gamma = id_A$
- *α is surjective*
- *γ is injective*

An abstract domain A is *trivial* if it is isomorphic to C, or if it is a singleton (i.e, all elements are mapped to $\top$). In the first case, no abstraction is performed, while in the second case there is no distinction. An abstract domain A is *strict* when $\gamma(\bot_A) = \bot_C$ and a concrete value $c \in C$ is *expressible* in A when $\gamma\alpha(c) = c$.

2.1.2 Abstract Semantics

For a target language L, many semantics can be defined. In static analysis, defining a **standard semantics** determines a **property** of the program executions. For abstract interpretation, we are interested in the **strongest** static property of interest. Starting with the work of Cousot, this is defined by the so-called **collecting semantics**. Collecting semantics are a starting point to define an entire class of static analyses, all the ones that approximate the strongest property. For our discussion, we will simply think of the target semantics in terms of fixpoints of a target function $f : C \to C$, on a concrete domain C. The interested reader can explore more deeply the topic in the provided references [Cou21].

The question now is how to define the abstract counterpart of the target semantics. Given a Galois connection (C, A, α, γ), we want to define an abstract function $f^\# : A \to A$ that is *correct* (or sound).

Definition 2.8. (Correct approximation) A function $f^\# : A \to A$ is a *correct* approximation of $f : C \to C$ if

$$\alpha f \sqsubseteq_A f^\# \alpha$$

Correctness transfers naturally to fixpoints when least fixpoint exists. That is, when $f^{\#}$ is correct, $\alpha(\text{lfp}(f)) \sqsubseteq_A \text{lfp}(f^{\#})$ holds. Recalling the previous section, the relation $\sqsubseteq_A$ encodes a loss of precision in the abstract domain. Thus, we can define the **best** correct approximation of f by simply composing it with α and γ.

Definition 2.9. (Best correct approximation). Given $f : C \to C$, the best correct approximation (bca) of f in A is defined as:

$$f^A = \alpha \circ f \circ \gamma$$

Any other abstract function $f^{\#} : A \to A$ is a correct approximation of f iff $f^A \sqsubseteq_A f^{\#}$, i.e, $f^{\#}$ is less precise than f^A.

Correctness encodes the property of being a safe over-approximation of the concrete behavior. Another desirable property of an abstraction is *completeness.*

Definition 2.10. (Completeness). Given a function $f : C \to C$, the abstract function $f^{\#} : A \to A$ is a *complete approximation* of f if

$$\alpha \circ f = f^{\#} \circ \alpha$$

holds.

An abstract domain A is a *complete abstraction* for f if there exists a complete approximation $f^{\#} : A \to A$ of f. Completeness represents the greatest achievable precision when abstracting the concrete semantics of f in A. In a complete approximation $f^{\#}$, the only loss of precision is due to the abstract domain itself, and not the abstract function. Completeness transfers to fixpoints like soundness, meaning that for a complete abstraction $f^{\#}$, the equivalence $\alpha(\text{lfp}(f)) = \text{lfp}(f^{\#})$ holds.

There exists an abstract function $f^{\#}$ such that completeness $\alpha \circ f = f^{\#} \circ \alpha$ holds iff $\alpha \circ f = \alpha \circ f \circ \gamma \circ \alpha$ iff $(\gamma\alpha) \circ f = (\gamma\alpha) \circ f \circ (\gamma\alpha) = (\gamma\alpha) \circ f \circ (\gamma\alpha) = \gamma \circ f^A \circ \alpha$. This means that, the possibility of defining a complete approximation $f^{\#}$ of f in A only depends on the bca f^A, i.e, completeness is a property of the abstract domain. When needed, we will use A in place of $\gamma \circ \alpha$. Completeness of f on A is denoted by $\mathbb{C}^A(f)$:

$$\mathbb{C}^A(f) \overset{\triangle}{\Longleftrightarrow} A \circ f = A \circ f \circ A = (\gamma \circ \alpha) \circ f \circ (\gamma \circ \alpha) = \gamma \circ f^A \circ \alpha$$

2.2 Term rewriting

Term Rewriting Systems (TRS) play a critical role in many areas of computer science, such as abstract data types, automated deduction systems, equational reasoning, and functional languages implementation. In this section,

we will introduce many of the basic concepts and properties of TRS. We will discuss Abstract Reduction Systems, signatures, general Term Rewriting, and strategic rewriting. The reader can consult the various books and papers for a more in-depth presentation of these arguments [BN99, KK90, Ohl02]. We introduce the topic with a basic example.

Example 2.11. Consider natural numbers defined by Peano axioms, with the constant 0 and the unary successor function s. Addition is defined by two identities:

$$0 + x = x$$
$$s(y) + x = s(y + x)$$

We can apply the identities to compute the sum of 1 ($s(0)$) and 2 ($s(s(0))$):

$$s(0) + s(s(0)) = s(s(0) + s(0)) = s(s(s(0)) + 0) = s(s(s(0)))$$

In this calculation, we are interpreting the identities as **rewrite rules** that transform a **sub-term** of a given **term** to another term. This brings us to the central notion, that is **term rewriting systems**. A term is built from **variables**, **constant symbols**, and **function symbols.** In the example, $+$ is a binary function, s is a unary function, 0 is a constant, and x and y are variables. Some terms built over these symbols are 0, x, $s(s(x))$, $y + s(0)$. Identities as shown before are symmetrical, meaning that we can apply them left to right or viceversa. This is only one possible instance of rewriting, where it is interpreted as a deduction system. More in general, rewriting can be seen as a computation mechanism where a rule $l \to r$ is used only in one direction. In a rewriting system, we usually want two main properties:

1. **Termination**. A term rewriting system is terminating if, given any initial expression, we can finitely apply rules until we reach a point where no more rules are applicable. This final expression is called **normal form**. The system in the previous example is terminating, but we can make it non-terminating by adding the rule $x + y = y + x$ encoding commutativity of addition. Then, we can create an infinite sequence $0 + x = x + 0 = 0 + x = \ldots$.

2. **Confluence**. A system is confluent if, given an initial term and two rewriting $t \to t_1,\ t \to t_2$, we can always find a term s such that $t_1 \to s$ and $t_2 \to s$. Intuitively, when a system is confluent, the order of rule application to a term t does not matter.

We now define precisely all the concepts mentioned.

2.2.1 Abstract Reduction Systems

Many of the more basic definitions and properties of rewriting can be defined in a more abstract setting, without first specifying the structure of terms of the target system. We start with defining these properties, on **Abstract Reduction Systems**.

Definition 2.12. An **Abstract Reduction System (ARS)** is a pair $(A, \rightarrow)$ where $\rightarrow$ is a binary relation on the set A, i.e, $\rightarrow \subseteq A \times A$. For pairs $(a, b) \in \rightarrow$ we write $a \rightarrow b$.

A finite or infinite sequence of reduction steps $a_0 \rightarrow a_1 \rightarrow \ldots \rightarrow a_n$ is called a *derivation*. Recall from set theory that, given two relations $R \subseteq A \times B$ and $S \subseteq B \times C$, their composition is defined by

$$R \circ S = \{(x, z) \in A \times C \mid \exists y \in B.\ (x, y) \in R \wedge (y, z) \in S\}$$

We want to compose reduction with itself to obtain a reduction sequence. We define the following notions:

$\xrightarrow{0}$	$:=$	$\{(x, x) \mid x \in A\}$	**Identity**
$\xrightarrow{i+1}$	$:=$	$\xrightarrow{i} \circ \rightarrow$	$i+1$-**fold composition** $i \geq 0$
$\xrightarrow{+}$	$:=$	$\bigcup_{i>0} \xrightarrow{i}$	**transitive closure**
$\xrightarrow{*}$	$:=$	$\xrightarrow{+} \cup \xrightarrow{0}$	**reflexive transitive closure**
$\leftarrow$	$:=$	$\{(y, x) \mid x \rightarrow y\}$	**inverse**
$\leftrightarrow$	$:=$	$\rightarrow \cup \leftarrow$	**symmetric closure**
$\xleftrightarrow{*}$	$:=$	$(\leftrightarrow)^*$	**symmetric transitive closure**

Some of these notions can also be expressed in terms of paths. For example, $x \xrightarrow{*} y$ if there is a finite path from x to y. The **closure** relations have a precise meaning. The P-closure of R is the least set with property P containing R. On this notation, we can add some terminology relative to the elements of A:

1. x is **reducible** iff there is a y such that $x \rightarrow y$. When $x \xrightarrow{+} y$, we call y a successor of x.

2. x is in **normal form** (irreducible) iff it is not reducible.

3. y is a normal form of x when $x \xrightarrow{*} y$ and y is irreducible. When the normal form of x is unique, we denote it by $x \downarrow$.

4. x and y are **joinable** iff there is a z such that $x \xrightarrow{*} z \xleftarrow{*} y$, and we write $x \downarrow y$ for this.

Now, we can formally define the central concepts:

Definition 2.13. A reduction $\rightarrow$ is called:

Church-Rosser	iff $x \xleftrightarrow{*} y \implies x \downarrow y$
confluent	iff $y_1 \xleftarrow{*} x \xrightarrow{*} y_2 \implies y_1 \downarrow y_2$
semi-confluent	iff $y_1 \leftarrow x \xrightarrow{*} y_2 \implies y_1 \downarrow y_2$
terminating	iff there is no infinite chain $a_0 \rightarrow a_1 \rightarrow \ldots$
normalizing	iff every element has a normal form
convergent	iff it is both confluent and terminating

$$\begin{tikzcd}
x \arrow[rr, leftrightarrow, "*"] \arrow[dr, dotted, "*"'] & & y \arrow[dl, dotted, "*"] \\
& z &
\end{tikzcd}
\qquad
\begin{tikzcd}
x \arrow[r, "*"] \arrow[d, "*"'] & y_1 \arrow[d, dotted, "*"] \\
y_2 \arrow[r, dotted, "*"'] & z
\end{tikzcd}
\qquad
\begin{tikzcd}
x \arrow[r] \arrow[d, "*"'] & y_1 \arrow[d, dotted, "*"] \\
y_2 \arrow[r, dotted, "*"'] & z
\end{tikzcd}$$

Figure 2.4: Church-Rosser property, confluence, and semi-confluence.

Figure 2.4 gives a visualization of three major properties. Note that the diagrams have a precise meaning: solid arrows represent universal and dotted arrows existential quantification.

Intuitively, semi-confluence looks weaker than confluence, but it turns out that the three properties are equivalent:

Theorem 2.14. *The following conditions are equivalent:*

1. $\rightarrow$ *is Church-Rosser.*

2. $\rightarrow$ *is confluent.*

3. $\rightarrow$ *is semi-confluent.*

2.2.2 Signatures and universal algebra

In the beginning of this section, it was stated that terms are built from variables, constant, and function symbols. The idea is formalized by taking

basic concepts from **universal algebra**. Universal algebra is a branch of mathematics that deals with the precise definition of algebraic structures. It defines both the syntactic aspects, but also their semantic models. For our purposes, we will explore only the syntactic level of such structures. More information on the topic can be found in various books and papers [Coh12, Grä08].

We start by defining **signatures**.

Definition 2.15. (Signature) A **signature** Σ is a set of function symbols, where each $f \in \Sigma$ is mapped to a non-negative integer n by the function $ar : \Sigma \to \mathbb{N}$, called the **arity** of f.

We denote by Σ_n the set of functions of a signature Σ having arity n. A signature can then be defined as an arity-indexed family of set of functions $\Sigma = \{\Sigma_n\}_{n \in \mathbb{N}}$. A symbol with arity 0 is called a *constant*.

Definition 2.16. Let Σ be a signature and X a set of **variables** such that $\Sigma \cap X = \emptyset$. The set $T_\Sigma(X)$ is defined inductively as:

- $X \subseteq T_\Sigma(X)$ (every variable is a term).
- $\Sigma_0 \subseteq T_\Sigma(X)$ (every constant is a term).
- for all $n > 0$, if $f \in \Sigma_n$, and $t_1, ..., t_n \in T_\Sigma(X)$, then $f(t_1, ..., t_n) \in T_\Sigma(X)$.

Given $t \in T_\Sigma(X)$, the set $Vars(t)$ is defined as the set of all the variables occurring in t. We can then define the set of **ground terms** T_Σ as the set of all terms that have no variables, i.e, $T_\Sigma = \{t \in T_\Sigma(X) \mid Vars(t) = \emptyset\}$.

Example 2.17. For Example 2.11, we define the signature $\Sigma = \{0,\ s,\ add\}$, where 0 is a constant, s is a function symbol with arity 1, and add is a binary constructor, defining addition between two numbers.

Then, we can define some terms such as:

- $0 \in T_\Sigma$
- $x \in T_\Sigma(X)$
- $s(add(x, s(s(0)))) \in T_\Sigma(X)$
- $add(s(0), s(s(0))) \in T_\Sigma$

The structure of a term is nicely illustrated with a tree representation, where nodes represent function symbols and arrows point to the argument of the function. Figure 2.5 depicts the term $s(add(x, s(s(0))))$.

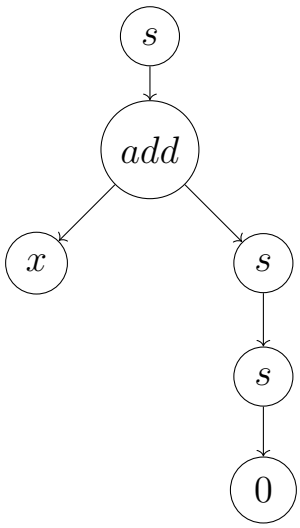


Figure 2.5: Tree representation of the term $t = s(add(x, s(s(0))))$.

By using the tree representation, we can refer to positions in a term. We do so by using strings of positive integers.

Definition 2.18. (Positions) Let Σ be a signature, X a set of variables, and $s \in T_\Sigma(X)$. The set of positions of s is a set $Pos(s)$ defined inductively as:

- $Pos(x) = \{\epsilon\}$, where ϵ is the empty string.
- $Pos(f(s_1, ..., s_n)) = \{\epsilon\} \cup \bigcup_{i=1}^{n} \{ip \mid p \in Pos(s_i)\}$.

The position ϵ is the **root position**, and it identifies the **root symbol** of s. The sub-term of s at position $p \in Pos(s)$ is defined as $s|_p$. Replacing a sub-term at position p with another term t yields a new term $s[t]_p$.

For Figure 2.5, $Pos(t) = \{\epsilon, 1, 11, 12, 121, 1211\}$ and $t|_{12} = s(s(0))$. A variable from the set X can be replaced with a term from $T_\Sigma(X)$ by using a **substitution**.

Definition 2.19. Let Σ be a signature and X a countable set of variables. A **substitution** is a function $\sigma : X \to T_\Sigma(X)$ such that $\sigma(x) \neq x$ for only finitely many variables. The domain of σ is defined as $Dom(\sigma) = \{x \in X \mid \sigma(x) \neq x\}$. If $Dom(x) = \{x_1, ..., x_n\}$, we write:

$$\sigma = \{x_1 \mapsto \sigma(x_1), ..., x_n \mapsto \sigma(x_m)\}$$

Any substitution σ can be inductively extended to an endomorphism $\hat{\sigma} : T_\Sigma(X) \to T_\Sigma(X)$ as follows:

- $\hat{\sigma}(x) = \sigma(x)$.

- $\hat{\sigma}(f(s_1, ..., s_n)) = f(\hat{\sigma}(s_1), ..., \hat{\sigma}(s_n))$.

For simplicity, we will not distinguish the notation and use σ for both functions. The application of σ to a term simultaneously replaces all variables in a term by their image in σ. As an example, let $t_1 = s(s(x) + s(y)), t_2 = s(0) + s(x + y)$ and let $\sigma = \{x \mapsto s(y), y \mapsto s(0)\}$. Then $\sigma(t_1) = s(s(s(y))) + s(s(0))$, and $\sigma(t_2) = s(0) + s(s(y) + s(0))$.

A term t is an **instance** of another term s iff there exists a substitution σ such that $\sigma(s) = t$.

Many sorted signatures The definition of signatures that we gave is actually that of **unsorted signatures**. **Many sorted signatures** are an extension where **sorts** are introduced to partition the domain of terms. Sorts are used to restrict the application of function symbols to specific sets of terms. To do so, the arity function is changed: each symbol f is associated to an arity $s_1 \times ... \times s_n \to s$ where $s_1, ..., s_n$ are the sorts of the arguments of f and s is the sort of the term constructed by applying f to its arguments.

Example 2.20. Take a signature $\Sigma = \{0,\ s,\ add,\ nil\ ,\ cons\}$ that represents lists of natural numbers. We can define two sorts $S = \{Nat,\ List\}$. The $cons$ symbols creates a new list by putting a natural number at the beginning of a list. Then its arity will be $Nat\ \times\ List\ \to\ List$.

To simplify reasoning, we will only deal with unsorted signatures, but note that the results that we will see are easily extended to the sorted case. A different matter would be to consider **order-sorted signatures**, another extension where sorts have a partial order relation associated.

2.2.3 Term rewriting systems

We can now formally define **term rewriting systems.** We start by explaining term identities and reduction between terms.

Definition 2.21. (Identities) Let Σ be a signature and X a set of variables. A Σ**-identity** (or identity) is a pair $(s, t) \in T_\Sigma(X) \times T_\Sigma(X)$. The identity (s, t) is written $s \approx t$, where s is the **left-hand side (lhs)** and t the **right-hand side (rhs)**.

Identities are used to transform terms into other "equivalent" terms by replacing the left-hand side by the right-hand side. This is exactly what we did in the introductory example, using the identities that define addition of natural numbers.

Definition 2.22. (Reduction) Let E be a set of identities over a signature Σ. The **reduction relation** $\rightarrow \subseteq T_\Sigma(X) \times T_\Sigma(X)$ is defined as:

$$s \rightarrow t \text{ iff } \exists (l, r) \in E, p \in Pos(s), \sigma \text{ substitution}. s|_p = \sigma(l), t = s[\sigma(r)]_p.$$

The reduction relation formally encodes the idea of rewriting a sub-term to an "equivalent" term. This situation is shown in Figure 2.6

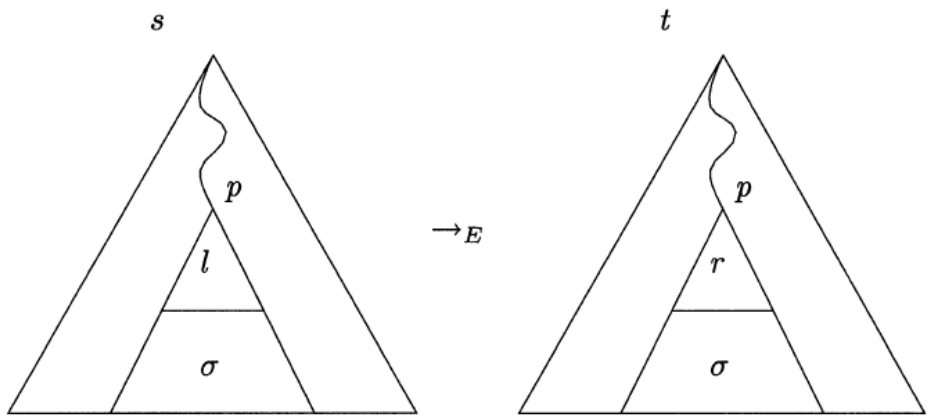


Figure 2.6: Rewriting of a sub-term at position p, using the equivalency (l, r) and the substitution σ.

Term rewriting systems are a special case of reduction systems, where identities are restricted to avoid certain sources of non-termination.

Definition 2.23. (Term rewriting). A **rewrite rule** is an identity $l \approx r$ such that l is not a variable and $Vars(l) \subseteq Vars(r)$. A **term rewriting system** is a pair $\mathcal{R} = (\Sigma, R)$ where Σ is a signature and R is a set of rewrite rules over Σ.

In many applications, term rewriting systems are extended with a set L of labels, which are names associated to rewrite rules. Rules are of the form $label : \ l \rightarrow r$. For example, we can name the rules for addition between naturals as:

- addZero : $add(0, x) \rightarrow x$
- addSucc: $add(s(x), y) \rightarrow s(add(x, y))$

In the following chapters, we will refer to labelled rewrite systems when not specified.

An instance of a lhs of a rule is called **redex** (reducible expression). **Contracting** a redex is the act of replacing it with the rhs of the rule.

Since any TRS $\mathcal{R}$ contains a set of identities, we can define the reduction relation $\rightarrow_{\mathcal{R}}$, that we call rewrite relation. We say that $\mathcal{R}$ is terminating, confluent, convergent, etc., if $\rightarrow_{\mathcal{R}}$ has the corresponding properties. When the context is obvious, we only write $\rightarrow$. It can be proven that all derived relation such as $\overset{*}{\rightarrow}, \overset{*}{\leftrightarrow}, \leftrightarrow$ are rewriting relations. Termination of a TRS is in the general case **undecidable**. This should not be of any surprise, as termination of rewriting and termination of programs are closely connected.

Unification The unification problem, in its simplest form (syntactic, first order unification) consists in finding a substitution ρ that identifies some terms pairwise. The substitution ρ is called *unifier*. Let us look at some simple examples:

$$\begin{aligned} f(x) =^? f(a) &\text{ has one unifier} \{x \mapsto a\} \\ x =^? f(y) &\text{ has many unifiers: } \{x \mapsto f(y)\}, \{x \mapsto f(a), y \mapsto a\}, ... \\ f(x) =^? g(y) &\text{ has no unifier} \end{aligned}$$

These examples shows that, in general, the problem may have zero, one, or many solutions. However, we want to find the best one, which in this case means finding **the most general unifier**.

Definition 2.24. A substitution σ is **more general than** σ' if there is a substitution δ such that $\sigma' = \delta\sigma$.

The "more general than" relation defines a **pre-order**, meaning that it is transitive and reflexive but not antisymmetric. Now, we can describe the unification problem in terms of **finding the most general unifier**.

Definition 2.25. (Unification.) A **unification problem** is a finite set of equations $S = \{s_1 =^? t_1, ..., s_n =^? t_n\}$. A **unifier (solution)** of S is a substitution σ such that $\sigma s_i = \sigma t_i$ for $i = 1, ..., n$. We call $U(S)$ the set of all unifiers of S. S is **unifiable** if $U(S) \neq \emptyset$. A substitution σ is a **most general unifier (mgu)** of S if σ is a least element of $U(S)$:

- $\sigma \in U(S)$
- $\forall \sigma' \in U(S)$, σ more general than σ'.

Rules can be generalized to unification instead of matching for left-hand sides, rewriting terms containing free variables instead. In this case, we talk about **narrowing.**

2.2.4 Strategic rewriting

Normalization of a term is a highly nondeterministic process: at every step many rules could be applied. This can lead very easily to non-termination. For example, any rule of the form $x \to x$ produces an infinite reduction. While this is a trivial example, infinite reductions can be much harder to work around, as they may be caused by circular rules. Take the same TRS describing natural numbers from before, and add the rules

$$\text{setZero} : s(x) \to 0$$
$$\text{toOne} : 0 \to s(0)$$

For any $n > 0$, we obtain an infinite reduction $n \to 0 \to s(0) \to 0 \ldots$. It was mentioned before that proving termination is in general undecidable, and as such we cannot rely on automatic verification to spot problematic systems. This is a relevant issue for the implementation of term rewriting systems. In a concrete implementation, it is desirable to have some form of control over the rewriting process. Straightforward rule application is too limiting: not only it easily leads to non-termination, but it is also not very efficient. A formal mechanism is needed, for instance, to specify the order for the search of solutions, to check some external conditions, to process sub-terms in a specific sequence, etc. In the last decades, a lot of work has been put towards the correct definition of these mechanisms, leading to the notion of **strategy** and **strategy languages**.

A strategy is an algorithm that controls how an input term is transformed. It can succeed, producing an output term (usually the normal form of the input) or fail. The simplest strategy possible is, obviously, **rule application**. We can see rules as *local transformations*, and strategies as the controlling of rule application.

Different definitions of strategies have been given in the rewriting community. We can first consider abstract strategies, defined on abstract reduction systems. In this setting, two different types of strategies are defined:

- **Extensional strategies** are defined as a subset of the set of all rewriting derivations. Formally, given an **ARS** $A = (S, \to)$, we denote by Γ_A^* the set of finite derivations, by Γ_A^ω the set of infinite derivations, and by $\Gamma_A = \Gamma_A^* \bigcup \Gamma_A^\omega$ the set of all derivations. An extensional strategy is a set $E \subseteq \Gamma_A$.

- **Intensional strategies** are defined as partial functions $\lambda : S^* \to \mathcal{P}(S \cup \{\top\})$ that associate each step in the rewriting sequence to the possible next steps. The symbol $\top$ is added to indicate that the derivation can stop at that point.

Extensional strategies represent an abstract execution as a whole, and are also useful to handle infinite sequences in a compact way. For these properties, they have been used in the context of model checking for strategy guided rewriting systems [RMOPV22].

On the other hand, the intensional approach gives a lot more control over the execution. At every step, a precise choice is made between the next steps to obey the guiding strategy.

In terms of expressiveness, it is always possible to define an extensional strategy from an intensional one, while the converse is not true. That is, extensional strategies are more expressive than intensional ones [BCDK10].

Strategy languages

Abstract strategies are a good formal starting point, but they lack in flexibility: a nice feature would be to define custom strategies depending on the context. In the 1990s, the idea of **strategy languages** started to came up. In these languages, rules are considered the primitive expressions, and additional constructs and combinators are provided to create more complex strategies. Essentially, we build a **program** that, given an initial term t and a set of rules R, rewrites t by choosing each step according to a **strategy expression** S. Each derivation is viewed as a possible computation that can be selected or discarded.

Many strategy languages have been proposed, such as **Elan** [BKK+98, BP97], **Stratego** [Vis01, Vis04] or the one in **Maude** [RMOPV18, CDE+20].

In the following, we will focus on the common aspects, briefly mentioning specific details when needed. A comprehensive description of such languages can be found in the provided references.

Elementary strategies are the basis of all languages. **Idle** and **Fail** are two elementary strategies that denote constant success and failure respectively. They can be encoded as rules Idle : $x \to x$ and Fail : $x \to KO$, where KO is a special symbol denoting failure. Rewriting rules such as *rl* : $l \to r$ are another basic expression. In many languages, rules are a builtin construct, but this is not necessary. **Stratego** defines rules in terms of two simpler primitives: **matching and building**. Matching $?p$ takes a term t, a pattern $p \in T_\Sigma(X)$, and a substitution σ and checks if $\sigma(p) = t$. Pattern building $!p$ is a sort of inverse operator: it builds the term $\sigma(p)$, using the current bindings in σ. Stratego decomposes each rule $l \to r$ into the program $?l; !r$, where ; is the **composing strategy**. Pattern matching is usually present in other languages with the more descriptive $match(p)$ definition.

Composing strategies is another fundamental aspect present in all languages. A derivation is built by successive compositions of a strategy s_1 followed by a strategy s_2. It can be found in different syntaxes: s_1 ; s_2, $seq(s_1, s_2)$, $s_1 <; s_2$ and so on.

Control flow combinators select different branches in the reduction sequence. A typical combinator is deterministic choice: s_1 <+ s_2 (or *first(*s_1, s_2*)*) selects the first non-failing strategy, prioritizing the left side and failing when both sides fail. **Elan** has also the nondeterministic variant $dc(s_1, s_2)$, which does not prioritize either side. A possible derived combinator is try(s) $= s$ <+ idle, that tries s but never fails. Conditionals like **if** S **then** S_1 **else** S_2 are present as well. As a derived operator, we can define **not(**S**)** = **if** S **then** fail **else** idle.

Recursive and iterative strategies allow the repeated application of an expression. Recursion is introduced both with named strategies referring to themselves, and with fixpoint operators. In Stratego, the fixpoint $recx(S)$ applies S to the entire term and $recx(S)$ to each occurrence of x inside the term. This allows the definition of derived iterators such as:

$$repeat(S) = recx(try(S; x))$$
$$while(c, S) = recx(try(c; S; x))$$

Traversal combinators control **at which position** of the target term a strategy s is applied. The combinators provided depend on the interpretation of each language. In Maude, the default interpretation is that a strategy expression can be applied at any position of a term. To control this, a **top** combinator is provided to restrict application only at the root of the term. Maude also provides different combinators to rewrite sub-terms with different strategies. In other languages like Stratego, the default interpretation is to allow application only at the root of the term. To iterate on sub-terms, typically two basic constructs are provided: **all(**s**)** applies s to each direct sub-term, while **one(s)** succeeds if there is at least one child on which s does not fail. From these basic combinators, more complex strategies can be

built:

$$
\begin{aligned}
TopDown(s) &= recx(s; all(x)) \\
BottomUp(s) &= recx(all(x); s) \\
OnceBu(s) &= recx(first(one(x), s)) \\
OnceTd(s) &= recx(first(s, one(x)))
\end{aligned}
$$

Chapter 3

Strat language

In this chapter, we present the strategy language that will be the focus of our analysis and verification. We discuss the main primitive constructs and present their semantics. Then, we show some example programs, and how to derive higher level constructs from basic ones.

3.1 The language

We consider a strategy language **Strat**, parametric to a term rewriting system $\mathbf{TRS} = (\mathbf{\Sigma}, \mathbf{L}, \mathbf{R})$. For simplicity, all the rules are assumed to be unconditional.

Strat takes ideas and constructs from other existing strategy languages (see Section 2.2.4), but there is a key difference. Usually, executing a strategy program means reducing an input term by evaluating a composed expression that guides the application of the defined rewrite rules, producing a result term or failing. We take instead a different approach. Our strategy programs modify and update **stores of terms**, where a store is intended as an environment that binds variables from a set X to the set T_Σ. Recall that T_Σ is the set of all ground terms constructed on the signature Σ, that is, terms that do not contain any variable. For terms that may contain variables, we use the set $T_\Sigma(X)$ instead. This choice is unconventional and forced us to rethink the typical approach to rewriting strategies, but it brings a key advantage: having a store-based semantics simplifies the application of the proof system that we will show in the next chapter.

Figure 3.1 shows the syntax of **Strat**. Strategy expressions ($sexp$) are the basic elements of the language. They apply a strategy operator sop to a pattern $p \in T_\Sigma(X)$ and produce a result term $t \in T_\Sigma$. The application of an operator is explicitly represented with the symbol **@**. Strategy commands

can add new variables, call named strategies and control the flow of execution. Their meaning is discussed in Section 3.2.2.

$$
\begin{array}{rll}
sop ::= & \mid\ id \ \mid\ fail & \\
& \mid\ !q & q \in T_\Sigma(X) \\
& \mid\ not(sop) & \\
& \mid\ ?q & q \in T_\Sigma(X) \\
& \mid\ rlabel & rlabel : l \to r \in R \\
& \mid\ sop \text{ » } sop & \\
& \mid\ sop \text{ <+ } sop & \\
& \mid\ f(sop_1, ..., sop_n) & f \in \Sigma_n \\
\\
sexp ::= & \mid\ sop@p & \\
\\
scom ::= & \mid\ \text{skip} & \\
& \mid\ \text{test } sexp & \\
& \mid\ x := sexp & x \in X \\
& \mid\ sexp \text{ <?> } p & p \in T_\Sigma(X) \\
& \mid\ strat(y) & strat(x) = scom \in Defs \\
& \mid\ scom; scom & \\
& \mid\ scom \oplus scom & \\
& \mid\ scom^* &
\end{array}
$$

Figure 3.1: Syntax of the strategy language.

3.2 Semantics

Semantics are defined by structural recursion on the elements of the language. A term store is denoted by a **ground substitution** $\theta : X \to T_\Sigma$, mapping variables to ground terms constructed on the signature Σ. The

special symbol KO (with arity 0) represents strategy failure, and is added to Σ. If a variable x is not present in the store, θ returns KO ($\theta(x) = KO$). $Dom(\theta)$ is the finite set of variables that θ does not map to KO: $Dom(\Theta) = \{x \in X \mid \theta(x) \neq KO\}$. The empty store is denoted by $\mathcal{E}$ ($Dom(\mathcal{E}) = \emptyset$). A substitution θ is extended to patterns $p \in T_\Sigma(X)$, by replacing each variable x in p with $\theta(x)$:

$$\theta(f(t_1, ..., t_n)) = f(\theta(t_1), ..., \theta(t_n))$$

It follows that, for a ground term t, $\theta(t) = t$.

Store update is defined as usual:

$$\theta[x \mapsto t](y) = \begin{cases} t & \text{if } y = x \\ \theta(y) & \text{otherwise} \end{cases}$$

A **variable replacement** $\theta[y/x]$ is a shorthand for assigning the value of x to y and removing x from the domain:

$$\theta[y/x](z) = \begin{cases} \theta(x) & \text{if } z = y \\ KO & \text{if } z = x \\ \theta(z) & \text{otherwise} \end{cases}$$

In the following, we will reuse the same notation also on sets of term stores. Given S, we denote by $S[x \mapsto t]$ the set $\{\theta[x \mapsto t] \mid \theta \in S\}$, and by $S[y/x]$ the set $\{\theta[y/x] \mid \theta \in S\}$. The pointwise definition of variable replacement for sets makes the following two properties true:

$$(S_1 \cup S_2)[x/y] = S_1[x/y] \cup S_2[x/y] \tag{3.1}$$

$$S_1 \subseteq S_2 = S_1[x/y] \subseteq S_2[x/y] \tag{3.2}$$

3.2.1 Expressions

Expressions are the core of our language. They guide the rewriting of an input term via different operators. Basic expressions can be composed through various combinators to form more complex ones. Their semantics is defined by a function $[\![_]\!]_ : sexp \to \Theta \to T_\Sigma$ (see Figure 3.2). An expression consists in the application of a **strategy operator** to an input term. The semantics of each operator is shown in Figure 3.3. An operator takes the pair (t, θ) as input. The term t is the one to transform, while θ is the environment storing all the needed binding to correctly evaluate an expression. The idea of defining the semantics in terms of pairs term-environment is taken from

$$
\begin{array}{rcl}
[\![_]\!]_ & : & sexp \to \Theta \to T_\Sigma \\
\\
[\![sop@p]\!]\theta & = & t \text{ where } [\![sop]\!](\theta(p), \theta) = (t, \theta)
\end{array}
$$

Figure 3.2: Semantics of expressions.

System S [V+98] and its derived languages. However, differently from there, our expressions cannot modify the environment. In **Strat**, new bindings are introduced only by commands. On one hand, this simplified the definition of the semantics, since a complex expression does not have to account for possible environment changes happening in its sub-expression. On the other hand, this choice made it harder to define pattern matching and rule application. A pattern $p \in T_\Sigma(X)$ can contain free variables not present in the current environment. These variables are used when we do not care about the value of a certain sub-terms. To ensure that matching works properly, we must extend the environment by binding these variables with the corresponding matching sub-terms. Our solution is to allow pattern matching to modify the environment, but only locally during the matching itself, while in the final result we always return the global environment θ.

We now briefly comment each operator.

Id and Fail always return the input term and KO respectively.

A constant pattern !q returns the evaluation of q in θ. Having a constant pattern as an operator is useful for passing it to the various combinators. For example, an expression such $\mathbf{s_1}$ **<+ !q** would use the constant pattern to return a default value q when s_1 fails.

Not is the negation of a strategy operator s. It returns the input term t when s fails on t, and KO when it succeeds.

Pattern matching is done via the **?** operator. Its semantics are defined in terms of $match$(Figure 3.4). The function $match(q, t, \theta)$ tries to match t with q by extending the bindings in θ with the free variables from $Vars(q)$. If the two terms matches, it returns the extended environment. The expression $?q@t$ succeeds when $match(q, t, \theta)$ is not equal to the empty store $\mathcal{E}$.

$$
\begin{array}{rcl}
[\![_]\!]_ & : & sop \to T_\Sigma \times \Theta \to T_\Sigma \times \Theta \\
\\
[\![id]\!](t,\theta) & = & (t,\theta) \\
[\![!q]\!](t,\theta) & = & (\theta(q),\theta) \\
[\![fail]\!](t,\theta) & = & (KO,\theta) \\
[\![not(s)]\!](t,\theta) & = & \begin{cases} (KO,\theta) & \text{if } [\![s]\!](t,\theta) \neq (KO,\theta) \\ (t,\theta) & \text{o.w} \end{cases} \\
[\![?q]\!](t,\theta) & = & \begin{cases} (t,\theta) & \text{if } match(q,t,\theta) \neq \mathcal{E} \\ (KO,\theta) & \text{o.w} \end{cases} \\
[\![rlabel]\!](t,\theta) & = & (\theta'(r),\theta) \quad \text{where } match(l,t,\mathcal{E}) = \theta' \\
[\![s_1 \text{ » } s_2]\!](t,\theta) & = & \begin{cases} [\![s_2]\!][\![s_1]\!](t,\theta) & \text{if } [\![s_1]\!](t,\theta) \neq (KO,\theta) \\ (KO,\theta) & \text{o.w} \end{cases} \\
[\![s_1 \text{ <+ } s_2]\!](t,\theta) & = & \begin{cases} [\![s_1]\!](t,\theta) & \text{if } [\![s_1]\!](t,\theta) \neq (KO,\theta) \\ [\![s_2]\!](t,\theta) & \text{o.w.} \end{cases} \\
[\![f(s_1,...,s_n)]\!](t,\theta) & = & congr(f(s_1,...,s_n),t,\theta)
\end{array}
$$

Figure 3.3: Semantics of operators.

Rules can be called by their label $rlabel$. The application of a rule $rlabel : l \to r$ is decomposed in two operations:

1. First, we have to match the input term t with the left-hand side l. To do so, we reuse the predicate $match$ previously defined.

2. When the match succeeds, we have to evaluate the right-hand side r in the extended store.

The patterns $l, r \in T_\Sigma(X)$ could contain variables which are also defined in the store θ. To avoid name clashes and unexpected behavior, pattern matching is done on an empty store $\mathcal{E}$, local to the rule itself. This local empty

$$match : T_\Sigma(X) \times T_\Sigma \times \Theta \to \Theta$$

$$match(x, t, \theta) = \begin{cases} \theta[x \mapsto t] \text{ if} & x \notin Dom(\theta) \\ \theta \text{ if} & \theta(x) = t \\ \mathcal{E} \text{ o.w.} & \end{cases}$$

$$match(f(t_1, ..., t_n), f(t'_1, ..., t'_n), \theta_0) = \theta_n$$
$$\text{if } match(t_i, t'_i, \theta_{i-1}) = \theta_i \neq \mathcal{E} \text{ for } i \in [1, n]$$

$$match(p, t, \theta) = \mathcal{E} \text{ o.w}$$

Figure 3.4: Match function.

store is extended to a new store θ', by binding the variables in l to the corresponding sub-terms in t, thanks to $match$. The right-hand side r is then evaluated in θ'. The final result will be $(\theta'(r), \theta)$. We return θ to maintain store immutability inside an expression, as previously stated.

Sequential composition is achieved by the » combinator. The expression $(s_1 \text{ » } s_2)@p$ only evaluates s_2 on the result of s_1 when s_1 does not fail on p.

The <+ combinator is a choice operator that prioritizes the left-hand operator. In other languages it can be found as the **first** combinator. Its meaning is that s_1 <+ s_2 is evaluated to s_2 only on terms on which s_1 fails.

A congruence operator applies an operator to each direct sub-term of a specific function symbol in Σ. For each $f \in \Sigma$, there is a corresponding operator $f(s_1, ..., s_n)$, which is applied to terms of the form $f(t_1, ..., t_n)$ by evaluating a strategy s_i on its corresponding sub-term t_i. Congruence fails when the application of one strategy fails or if the root symbol of the operator and that of the term do not match. The formal definition is given by the *congr* function, in Figure 3.5

$$congr(f(s_1, ..., s_n), f(t_1, ..., t_n), \theta) = (f(t'_1, ..., t'_n), \theta)$$
$$\text{if } (t'_i, \theta) = [\![s_i]\!](t_i, \theta) \wedge ti' \neq KO \text{ for } i \in [1, n]$$

$$congr(f(s_1, ..., s_n), g(t_1, ..., t_m), \theta) = KO$$
$$\text{o.w}$$

Figure 3.5: Congruence function.

3.2.2 Commands

Commands are defined on the powerset of stores $\mathbb{S} : \mathcal{P}(\Theta)$. We overload the notation and define the function $[\![_]\!]_ : scom \to \mathbb{S} \to \mathbb{S}$.

Figure 3.6 presents the semantics of each command.

The basic command **skip** leaves a set of stores S unchanged. A variable assignment $x := sexp$ evaluates $sexp$ and then binds the result to the name x. The guard **test** $sexp$ filters all the stores on which $sexp$ does not fail. Sequential composition $c_1; c_2$ evaluates c_1 and then c_2. The command $c_1 \oplus c_2$ is a nondeterministic choice, and the result is given by the union of the results of c_1 and c_2 . The term c^* is the Kleene iteration of c, where c is executed 0 or any bounded number of times in sequence.

The <?> command has a similar purpose of the ? operator. It tries to match a pattern p with the term returned from executing an expression $sexp$. Differently from an expression $?p$, a command $sexp$ <?> p keeps the extended bindings, updating θ accordingly. A match can be used to easily bind fresh variables to specific sub-terms and to branch out the execution based on the shape of a given term.

Named strategies introduce the possibility of calling a strategy s by referring to its associated name $strat$, passing a variable y as a parameter. We assume the existence of a set $Defs$, storing all the strategy definitions. Adding named strategies was needed to handle the recursive nature of algebraic terms. Since we are dealing with trees of arbitrary depth, simple iteration with the c^* operator is not enough to effectively traverse a term. A named strategy can refer to itself and, combined with other commands,

$$
\begin{array}{rcl}
[\![_]\!]_ & : & scom \to \mathcal{P}(\Theta) \to \mathcal{P}(\Theta) \\
Defs & = & \{strat_i(x) = scom_i\}_{i \in I} \\
[\![\text{ skip }]\!]S & = & S \\
[\![\text{test } sexp]\!]S & = & \{\theta \in S \mid [\![sexp]\!]\theta \neq KO\} \\
[\![x := sexp]\!]S & = & \{\theta[x \mapsto [\![sexp]\!]\theta] \mid \theta \in S\} \\
[\![sexp \text{ <?> } p]\!]S & = & \{\theta' \mid \theta \in S,\ match(p, [\![sexp]\!]\theta, \theta) = \theta' \neq \mathcal{E}\} \\
[\![c_1; c_2]\!]S & = & [\![s_2]\!]([\![s_1]\!]S) \\
[\![strat(y)]\!]S & = & \begin{cases} call(scom, x, y, S) & \text{if } strat(x) = scom \in Defs \\ \emptyset & \text{o.w.} \end{cases} \\
[\![c_1 \oplus c_2]\!]S & = & [\![c_1]\!]S \cup [\![c_2]\!]S \\
[\![c^*]\!]S & = & \bigsqcup\{[\![c]\!]^n S \mid n \in \mathbb{N}\}
\end{array}
$$

Figure 3.6: Semantics of commands.

allows us to concisely access and modify sub-terms. The $call$ predicate from Figure 3.6 is defined as follows:

$$call(s, x, y, S) = S'[y/x][\overline{w}/\overline{z}] \text{ where } S' = [\![s]\!]S[x/y][\overline{z}/\overline{w}]$$

We now explain it in detail. The semantics of a call must deal with the following issues:

- Pass the argument y to the formal parameter x
- Ensure that the body s is executed in its own local scope, meaning that it can redefine variables present in the environment of the caller, without affecting the original state.

The replacement $[x/y]$ models the passing of the value of y to the formal parameter x. Scoping is achieved by the replacement $[\overline{z}/\overline{w}]$. We denote with $\overline{w}$ the set of variables bound in the environment of the caller that are different from y:

$$\overline{w} = \bigcup_{\theta \in S} Dom(\theta) \setminus \{y\}$$

Since we are dealing with set of stores, we consider the union of all domains minus y. The set $\overline{w}$ is replaced by a set of **fresh variables** $\overline{z}$. These new variables must not be defined in the original environment and must not be present in the body of s. Formally, given a call $[\![strat(y)]\!]S$:

$$\overline{z} \cap (\overline{w} \cup fv(s)) = \emptyset$$

The set $fv(s)$ defines by structural induction the set of free variables, those that are declared in the body of s and are different from the formal parameter x. The definition is straightforward, so we omit it. The goal of fresh variables is to save the original store to recover it after the call.

After execution, we go back to the original caller. In classical programming languages, this is handled by adding a **return** statement, producing a final value and returning it to the caller. Then, a call is evaluated like an expression, and the final value can be assigned to a variable. To refrain from further complicating the base language by adding more elements, we adopted the simpler approach of restoring the original environment θ, updating y to the final value of x after executing the body s. Essentially, y is used both as a parameter and as a return variable. To cope with this choice, strategy calls have a limitation: we can only pass a variable as an argument and not a generic expression. An alternative would be to reserve a special variable **res**, which stores the final result of the last executed call.

The last step is restoring the caller environment, by redefining all the variables different from y. This is done with the replacement $[\overline{w}/\overline{z}]$ in the final store S'.

Derived expressions and commands can be easily defined to encode complex behaviors without sacrificing readability.

The expression **try(**s**)** = s <+ id is a safer version of s, reverting to the input term when s fails; **check(**s**)** = **not(not(**s**))** can be thought as the opposite of try. It is used to test if s succeeds on a specific input without changing it. When s fails, **check(**s**)** fails too.

For self assignments, we can write

$$\mathbf{x\ @=\ s} \triangleq \text{x := sexp@x}$$

A direct pattern assignment $x :=!p@x$ is written as $x :=!p$.

Guarded branching and looping are encoded by the **if-then-else** and **while-do** commands.

$$b = sop@p \quad \neg b = not(sop)@p$$

$$\textbf{if } (b) \textbf{ then } c_1 \textbf{ else } c_2 \triangleq (\text{test } b; scom) \oplus (\text{test } \neg b; scom);$$

$$\textbf{while } (b) \textbf{ do } c \triangleq (\text{test } b; c)^*; \text{ test } \neg b;$$

They work similar to the homonymous constructs in standard programming languages. The difference is in the evaluation of the guard: since boolean values **true** and **false** are not part of the language, we use instead the KO symbol as the "false" value, and every other term as "true". As an example, we can define the program

$$\text{norm(x, s)} \triangleq \textbf{while } \text{s@x } \textbf{do } \text{x @= s;}$$

which applies the strategy expression s to x until it fails.

The case-of construct allows a program to branch based on the shape of a pattern $p \in T_\Sigma(X)$:

```
case p of
    | p1 -> c1
    | p2 -> c2
    ⋮
    | pn -> cn
    | else -> cn+1
```

The construct is just a clearer way of writing this command:

$$(p \text{ <?> } p1;\ c_1)\oplus$$

$$(p \text{ <?> } p_2;\ c_2)\oplus$$

$$\vdots$$

$$(p \text{ <?> } p_n;\ c_n)\oplus$$

$$(test\ not(?p_1 \text{ <+ ... <+ } ?p_n)@p;\ c_{n+1})$$

The <?> command binds the variables of each pattern, making them usable in the successive command. Take the following program:

```
case x of
  | f(y) -> y := !a;
  | g(y) -> y := !b;
  | else -> skip;
```

It gets de-sugared into:

```
(x <?> f(y); y := !a) ⊕
(x <?> g(y); y := !b) ⊕
(test not(?f(y) <+ ?g(y))@x; skip)
```

When the **else** branch does nothing, we will omit it entirely:

```
(x <?> f(y); y := !a) ⊕
(x <?> g(y); y := !b)
```

The syntactic sugar presented allows us to express complex operations in a simpler and clearer way, and for this, it will be used whenever possible.

We conclude the chapter with a final example, aimed at illustrating the expressiveness and functioning of the language.

Example 3.1. Assume that we have the following TRS:

$$\Sigma_0 = \{zero, true, false\}$$

$$\Sigma_1 = \{succ, even\}$$

$$\Sigma = \Sigma_0 \cup \Sigma_1 \cup \Sigma_2$$

$$\begin{aligned} R := \{ & \\ & evenZero : even(zero) \to true, \\ & evenSuccZero : succ(zero) \to false, \\ & evenSucc : even(succ(succ(n))) \to even(n) \\ \} & \end{aligned}$$

This system extends the standard TRS for natural numbers with the constants $true$ and $false$ and the predicate even, which checks if a number n is even. The predicate behavior is defined by the rules in R. The following strategy program tries to normalize $even(n)$ terms:

```
evenN ≝ evenZero <+ evenSuccZero <+ evenSucc
norm(evenN,n) =
    (test evenN@n;  n @= evenN)*;
     test not(evenN@n);
```

The expression $evenN$ chooses the first non-failing rule. We apply it to n iteratively until it fails, which signals that we reached a ground value $true$ or $false$ (or KO for ill-formed terms).

As an example, let's evaluate $norm(evenN, n)$ with an initial single store

$$\{n \mapsto even(4)\} = \{n \mapsto even(succ(succ(succ(succ(zero)))))\}$$

First, the loop is evaluated. Following the semantics from Figure 3.6, the kleene star command c^* collects **all** intermediate results of execution. For ease of exposition, we only show the last value produced by each iteration.

$$\begin{array}{l}
\{n \mapsto even(succ(succ(succ(succ(zero)))))\} \\
(test\ evenN@n;\ n@ = evenN) \\
\{n \mapsto even(succ(succ(zero)))\} \\
(test\ evenN@n;\ n@ = evenN) \\
\{n \mapsto even(zero)\} \\
(test\ evenN@n;\ n@ = evenN) \\
\{n \mapsto true\}
\end{array}$$

When $n \mapsto true$, the following $evenN$ calls fail on input $true$ and the loop terminates. The last command assures that the rules are not applicable anymore, ending with $n \mapsto true$ as expected:

$$\begin{array}{l}
\{n \mapsto true\} \\
(not(evenN@n)) \\
\{n \mapsto true\}
\end{array}$$

An alternative solution allows us to drop the $even$ constructor and its rules from the TRS, exploiting the constructs of $Strat$ instead:

```
even(x):
  case x of
    | zero -> x @= !true
    | succ(zero) -> x @= !false
    | succ(succ(m)) -> even(m); x:=!m
```

Let's evaluate even(x) with $x = 2 = succ(succ(zero))$. First,we desugar the case-of syntax:

$$\begin{array}{l}
(x \text{ <?> } zero;\ x := !true) \oplus \\
(x \text{ <?> } succ(zero);\ x := !false) \oplus \\
(x \text{ <?> } succ(succ(m));\ even(m);\ x := !m)
\end{array}$$

We can avoid evaluating the first two branches, since x does not match with the required pattern. In the third branch, x matches with $succ(succ(m))$, binding m to zero:

$$
\begin{array}{l}
\{x \mapsto succ(succ(zero))\} \\
(x \text{ <?> } zero;\ x :=!true) \\
\oplus \\
(x \text{ <?> } succ(zero);\ x :=!false) \\
\oplus \\
(\llbracket \mathbf{x} \text{ <?> } \mathbf{succ(succ(m))} \rrbracket;\ even(m); x :=!m) \\
\{x \mapsto succ(succ(zero)), m \mapsto zero\}
\end{array}
$$

The recursive call $even(m)$ is now executed. For brevity, only the "zero" branch evaluation is shown:

$$
\begin{array}{l}
\{x \mapsto succ(succ(zero)), m \mapsto zero\} \\
(\llbracket \mathbf{even(m)} \rrbracket) \\
\text{(start call)} \\
\{x \mapsto zero\} \\
(\llbracket \mathbf{x} \text{ <?> } \mathbf{zero;} \rrbracket;\ x :=!true) \\
\{x \mapsto zero\} \\
(\llbracket \mathbf{x} :=!\mathbf{true} \rrbracket) \\
\{x \mapsto true\} \\
\text{(finish call)} \\
\{x \mapsto succ(succ(zero)), m \mapsto true\}
\end{array}
$$

In each store, we only show the bindings relevant to the actual call and omit the variables redefined by the replacement $[\overline{z}/\overline{w}]$ in the semantics, to avoid cluttering the example. The recursive call terminates, restoring the previous store but with the update $m \mapsto true$. The final assignment is executed, obtaining $x \mapsto true$ as expected:

$$
\begin{array}{l}
\{x \mapsto succ(succ(zero)), m \mapsto true\} \\
(x \text{ <?> } succ(succ(m));\ even(m); \llbracket \mathbf{x} :=!\mathbf{m} \rrbracket) \\
\{x \mapsto true, m \mapsto true\}
\end{array}
$$

Chapter 4

Local Completeness Logic on a Strategy Language

In the previous chapter, we defined the strategy language **Strat**. In this one, we show how to formally verify programs of such language. The analysis is done by adapting Local Completeness Logic [BGGR21] to the strategic rewriting context. LCL is an abstract interpretation framework that tries to combine the typical over-approximation approach with the more recent under-approximated methods, in order to reduce false alerts. To do so, it relaxes the classical notion of global completeness for abstract interpretation, introducing the novel notion of **local completeness** instead. We present the main features of this framework and show how it was adapted for **Strat**.

4.1 Local completeness Logic

Local Completeness Logic is a logical proof system $\vdash_A$ parametrized by an abstraction A, which we will call LCL_A. LCL_A is a Hoare-like logic.

Hoare Logic [Hoa69] is a proof system where the statements of the logic are triples $\{P\}r\{Q\}$. P is the set of properties valid before executing a program r, and Q is the set of *post-conditions* that are proven in the logic by executing r. To make the proof system sound, Q is an over-approximation of the actual program behavior $[\![r]\!]$. Recall from Section 2.1.2 that, in abstract interpretation, $[\![r]\!]$ is the collecting semantics of the program, storing the **strongest property** (or post-condition) valid after the execution. In LCL_A, the statements are triples $\vdash_A [P]r[Q]$ asserting that:

1. Q is an *under-approximation* of $[\![r]\!]P$.

2. $[\![r]\!]P$ is **locally complete** for input P on A.

3. Q and $[\![r]\!]P$ share the same **over-approximation** in A.

Condition 2 introduces the central notion of the logic, namely **local completeness**.

Completeness, as defined in Section 2.1.2, is a *global* notion, meaning that it must hold for **all** input programs to be true. More precisely, given an abstraction A, a collecting semantics $[\![r]\!](P)$ of a target program r and a pre-condition P, we say that A is complete for c when:

$$\forall P, [\![r]\!]^{\#}_{A}\alpha(P) = \alpha([\![r]\!](P))$$

The notation $[\![r]\!]^{\#}_{A}$ represents the abstract semantics defined on A. The universal quantification makes it hard to achieve completeness, even for very simple programs.

Example 4.1. Consider an imperative program

$$\text{r} \triangleq \textbf{if } (x > 0) \textbf{ then } x = x - 3 \textbf{ else } x = -2x$$

A typical abstraction in this context is the **interval abstraction Int**. Instead of associating each variable to the set of all possible values that it holds, we only hold the minimum and maximum, representing the range of its concrete value. For example, the input $P = \{0, 2, 5, 10\}$ for x is abstracted to the range $[0, 10]$. The concretization function just maps the range to the set of elements contained by it. We show now that completeness does not hold for r w.r.t **Int**.

Recall that, for a program f and abstraction A, completeness means that

$$A \circ f = A \circ f \circ A$$

Now let's take r as our program, Int as our A and assume that we start with a pre-condition $P = \{0, 3\}$. For completeness to hold, we would need that

$$Int([\![r]\!]P) = Int([\![r]\!]Int(P))$$

For brevity, $[\![r]\!]Int(P)$ means that the program r is applied to the set of elements enclosed in the range $Int(P)$. Let's compute the final values to see that completeness is not true for P:

$$\begin{aligned} Int([\![r]\!]P) &= Int([\![r]\!](\{0,3\})) = Int(\{0,0\}) &&= [0,0] \\ Int([\![r]\!]Int(P)) &= Int([\![r]\!]Int(\{0,3\}) = Int(\{0,-2,-1\}) &&= [-2,0] \end{aligned}$$

Completeness is a desirable property, as it would mean that no false alert can be raised. The universal quantification often causes it to not hold, making the analysis incomplete and thus not reliable. Bruni et Al. [BGGR21] removed this global requirement for LCL, relaxing the notion to a **local** one instead.

Definition 4.2. (Local Completeness) An abstract domain $A \in Abs(C)$ is locally complete for $f : C \to C$ on a concrete value $c \in C$ if:

$$Af(c) = AfA(c)$$

We write $\mathbb{C}_c^A(f)$.

We can observe that local completeness is true for any f on any abstract value $A(c)$. Local completeness means that we can simply reason about *some* program traces, those induced by the input c. This is a weaker requirement, and it can bring completeness back to some cases where the classic version is not true.

Example 4.3. We consider again Example 4.1, and show that completeness is valid if we only take certain values for P. For example, take $P = \{0, 1, 3\}$:

$$\begin{aligned} Int([\![r]\!]P) = Int([\![r]\!](\{0,1,3\})) = Int(\{0,-2\}) &= [-2,0] \\ Int([\![r]\!]Int(P)) = Int([\![r]\!]Int(\{0,1,3\}) = Int(\{0,-2,-1\}) &= [-2,0] \end{aligned}$$

To explain Condition 1 and 3 we now present the full proof system.

LCL is defined over a language of *regular commands*:

$$\text{Reg} \ni \text{r} ::= \text{e} \mid \text{r}; \text{r} \mid \text{r} \oplus \text{r} \mid \text{r}^*$$

The reasoning behind this choice is for generality: regular commands are parametric on the basic functions (transfer expression) e $\in Exp$, which provides the basic commands. This allows the proof system to be applied to different contexts with little work. In fact, in the initial paper, the analysis was performed on a small imperative formal language that could express other well known imperative languages such as Imp [Win93].

The concrete semantics of the language $[\![\cdot]\!] : \text{Reg} \to C \to C$ are presented in Figure 4.1. It's assumed that an additive function $(|_|) : \text{Exp} \to C \to C$ on a complete lattice C is defined. The function characterizes the semantics of the basic transfer expression.

Since the logic is parametrized by an abstraction A, we must also delineate the abstract semantics, which are presented in Figure 4.2. The abstract semantics $[\![\cdot]\!]_A^\# : Reg \to A \to A$ are defined over the abstract domain

$$[\![e]\!]c \triangleq (|e|)c \qquad [\![r_1 \oplus r_2]\!]c \triangleq [\![r_1]\!]c \vee [\![r_2]\!]c$$
$$[\![r_1; r_2]\!]c \triangleq [\![r_2]\!]([\![r_1]\!]c) \qquad [\![r^*]\!]c \triangleq \bigvee\{[\![r^n]\!]c \mid n \in \mathbb{N}\}$$

Figure 4.1: Concrete semantics of regular commands.

$$[\![e]\!]^{\#}_A a \triangleq [\![e]\!]^A = (\alpha \circ [\![e]\!] \circ \gamma)a \qquad [\![r_1 \oplus r_2]\!]^{\#}_A a \triangleq [\![r_1]\!]^{\#}_A a \vee [\![r_2]\!]^{\#}_A a$$
$$[\![r_1; r_2]\!]^{\#}_A a \triangleq [\![r_2]\!]^{\#}_A([\![r_1]\!]^{\#}_A a) \qquad [\![r^*]\!]^{\#}_A a \triangleq \bigvee\{([\![r]\!]^{\#}_A)^n a \mid n \in \mathbb{N}\}$$

Figure 4.2: Abstract semantics of regular commands.

A. The concretization and abstraction functions γ and α are defined as expected.

We can finally explain how the proof system works. The rules are shown in Figure 4.3

$$\frac{\mathbb{C}^A_P(e)}{\vdash_A [P]e[[\![e]\!]P]}\ \text{(transfer)} \qquad \frac{P' \leq P \leq A(P') \quad \vdash_A [P']r[Q'] \quad Q \leq Q' \leq A(Q)}{\vdash_A [P]r[Q]}\ \text{(relax)}$$

$$\frac{\vdash_A [P]r_1[R] \quad \vdash_A [R]r_2[Q]}{\vdash_A [P]r_1; r_2[Q]}\ \text{(seq)} \qquad \frac{\vdash [P]r_1[Q_1] \quad \vdash [P]r_2[Q_2]}{\vdash_A [P]r_1 \oplus r_2[Q_1 \vee Q_2]}\ \text{(join)}$$

$$\frac{\vdash_A [P]\mathrm{r}[R] \quad \vdash_A [P \vee R]\mathrm{r}^*[Q]}{\vdash_A [P]\mathrm{r}^*[Q]}\ \text{(rec)} \qquad \frac{\vdash_A [P]\mathrm{r}[Q] \quad Q \leq A(P)}{\vdash_A [P]\mathrm{r}^*[P \vee Q]}\ \text{(iterate)}$$

Figure 4.3: The proof system LCL_A.

The unique rules are **(transfer)** and **(relax)**, as their premises depend on the underlying abstraction.

Rule (relax) is peculiar, as it combines over- and under-approximating reasoning. Condition Q is an under-approximation of the exact behavior, but its abstraction $A(Q)$ is a sound over-approximation. The logical orderings $P' \leq P$ and $Q \leq Q'$ are reversed w.r.t the typical ordering of Hoare Logic to ensure that the post-conditions Q are an under-approximation. The fundamental idea is that the under-approximation Q is constrained to have the same abstraction of the concrete behavior $[\![r]\!]P$, which preserves the precision of the analysis. On the technical side, the rule is valid thanks to

the following observation on local completeness: if A is locally complete for some $c \in C$, then A is locally complete for any $d \in C$ such that $c \leq d \leq A(c)$ [BGGR21] (Lemma V.1). The rule (transfer) checks that a basic expression e is locally complete for P, in which case it infers the semantics $[\![e]\!]$ as its post-condition. Rules (seq) and (comp) are defined as expected. The rule (rec) simply unfolds one step of the Kleene iteration, until (iterate) can be applied. (iterate) is similar to (relax), as it introduces an under-approximating post-condition based on A as well. The concrete behavior $[\![r^*]\!]$ can lead to non-termination, but (iterate) can exploit A to stop when the abstraction of P is already an abstract invariant, consequence of the premise $Q \leq A(P)$. This returns an under-approximation of the concrete infinitary invariant.

4.2 LCL on Strat

We now investigate how to interpret the LCL proof system on Strat. In order to reach the wanted result, we must tackle the following issues:

- Define the notion of abstract domain for a Strat program and the predicates that we want to prove.
- Ensure that the abstract semantics defined for LCL are a sound approximation of the concrete semantics of Strat programs.
- Adapt the proof system to attain valid proofs for Strat .

4.2.1 Program Properties for Strat

Since our collecting semantics are defined on the powerset of term stores $\mathcal{P}(\Theta)$, we must define a valid **store abstraction**. A store abstraction uses an appropriate value abstraction for each variable present in the store. In literature, store abstractions are traditionally divided in two groups: **non-relational** and **relational** domains. A non-relational domain treats each variable separately, and cannot express relations between them. In a **relational** domain, variables can be coupled by relations, which makes the analysis more expressive but also computationally more expensive. In the following, we will focus on non-relational domains only. We proceed with formalizing the idea, by following a variation of the approach by Giacobazzi et al. [GLR15]. First, we represent a store $\theta \in \Theta$ as the cartesian product

$$\theta = \prod_{x \in X} \theta_x$$

for a given set of variables X. An element θ_x is a tuple $(x, \theta(x))$, mapping each variable x to its concrete value $\theta(x)$. For our purposes, we consider the case where values are only taken from one set of values T_Σ, but this can be easily extended to the case where each variable has its own concrete domain. Now, our concrete domain is $\mathcal{P}(\Theta)$. An abstract domain $A \in Abs_{\alpha,\gamma}(\mathcal{P}(\Theta))$ is non-relational when it can be represented as

$$A = \prod_{x \in X} A_x$$

For each variable x, a Galois connection ($\mathcal{P}(T_\Sigma)$, α_x, γ_x, A_x) exists between the concrete domain for values and an abstract domain A_x. When working with the same abstract domain for each variable, we will omit the subscript x. The Galois connection ($\mathcal{P}(\Theta)$, (α, γ), A) is defined by:

$$\alpha(S) = \prod_{x \in X} \alpha_x(\pi_x(S))$$
$$\gamma(a) = \prod_{x \in X} \gamma_x(\pi_x(a))$$

The order relation in the abstract is defined as

$$a_1 \leq_A a_2 \text{ iff } \forall x \in X. \pi_x(a_1) \leq_{A_x} \pi_x(a_2)$$

Both the concretization and abstraction are performed **independently** on every variable, thus preventing the possibility of expressing relations between them. The projection function $\pi_x : \Theta \to T_\Sigma$, when defined on a single store θ, just returns the value stored in x ($\pi_x(\theta) = \theta(x)$). We overload the function, defining the additive extension to sets of stores $\pi_x : \Theta \to T_\Sigma$. In this case π_x returns the union of all values of x in a set S ($\pi_x(S) = \{\theta(x) \mid \theta \in S\}$). Projection on an abstract store a is analogous.

A property specification $Spec$ will express properties about variables. In the simplest case, we can reason about a single variable x. Then, $Spec$ will represent all the stores in which $\theta(x)$ satisfies the target property.

4.2.2 Adapting proofs

The design of Strat simplified the interaction with LCL. Thanks to the general nature of the logic, adapting the proof system was for the most part straightforward. If we ignore strategy calls for the moment, we can see that every other strategy command can be mapped directly to an LCL rule. Basic transfer expressions e $\in Exp$ become the **skip**, **test**, **variable assignment**

and **matching** commands. Then, the rule (transfer) can be stated symbolically as follows:

$$\frac{}{\vdash_A [P]\ \text{skip}\ [P]}\ \text{(skip)} \qquad \frac{\mathbb{C}_P^A([\![\text{test } s]\!])}{\vdash_A [P]\text{test } s[P \wedge s \neq KO]}\ \text{(test)}$$

$$\frac{\mathbb{C}_P^A([\![x := s]\!])}{\vdash_A [P]x := s[\exists t.P[t/x] \wedge x = s[t/x]]}\ \text{(assign)} \qquad \frac{\mathbb{C}_P^A([\![s \text{ <?> } p]\!])}{\vdash_A [P]s \text{ <?> } p[\exists \overline{t}.P[\overline{t}/\overline{x}] \wedge s[\overline{t}/\overline{x}] = p]}\ \text{(match)}$$

We assume to have an assertion language to define pre- and post-conditions, where equalities between basic expression can be combined through the standard logical connectives and quantifiers. The basic expressions are the ones from Strat. **Logical variables** z are introduced together with normal variables to evaluate logical expressions.

Rules skip and assign are standard for Hoare triples. The test rule needs to account for the absence of boolean values from the language. Because of this, we cannot state $[P \wedge s]$ as the post-condition, but instead add the assertion $x \neq KO$.

The **(match)** rule is similar to variable assignment, but for term patterns $p \in T_\Sigma(X)$. We denote by an indexed set $\overline{x}$ the set $Vars(p)$. The simultaneous substitution $[\overline{t}/\overline{x}]$ replaces each $x_i \in \overline{x}$ with the corresponding ground term t_i.

Strategy combinators $\mathbf{c_1}; \mathbf{c_2}, \mathbf{c_1} \oplus \mathbf{c_2}$, and $\mathbf{c}^*$ are handled directly by the **(seq), (join), (iterate), and (rec)** rules:

$$\frac{\vdash_A [P]c_1[R] \quad \vdash_A [R]c_2[Q]}{\vdash_A [P]c_1; c_2[Q]}\ \text{(seq)} \qquad \frac{\vdash [P]c_1[Q_1] \quad \vdash [P]c_2[Q_2]}{\vdash_A [P]c_1 \oplus c_2[Q_1 \vee Q_2]}\ \text{(join)}$$

$$\frac{\vdash_A [P]\text{c}[R] \quad \vdash_A [P \vee R]\text{c}^*[Q]}{\vdash_A [P]\text{c}^*[Q]}\ \text{(rec)} \qquad \frac{\vdash_A [P]\text{c}[Q] \quad Q \leq A(P)}{\vdash_A [P]\text{c}^*[P \vee Q]}\ \text{(iterate)}$$

Following a similar reasoning, the abstract semantics from Figure 4.2 are taken as is for **Strat**. The abstract domain A is a generic abstraction defined as in Section 4.2.1, for the concrete domain $C = \mathcal{P}(\Theta)$. Again, procedure calls are not considered right now, as they will be dealt with in the next section.

LCL works with the assumption that the concrete semantics for e $\in Exp$ are defined by an **additive** function. We will prove in the following sections that the assumption is valid for **Strat**.

4.2.3 Procedure calls

The addition of calls allows **Strat** to be as expressive as needed, but it also poses a challenge for its verification. The initial presentation of LCL did not

$$[\![e]\!]^{\#}_A a \triangleq [\![e]\!]^A = (\alpha \circ [\![e]\!] \circ \gamma)a \qquad [\![c_1 \oplus c_2]\!]^{\#}_A a \triangleq [\![c_1]\!]^{\#}_A a \vee [\![c_2]\!]^{\#}_A a$$
$$[\![c_1; c_2]\!]^{\#}_A a \triangleq [\![c_2]\!]^{\#}_A([\![c_1]\!]^{\#}_A a) \qquad [\![c^*]\!]^{\#}_A a \triangleq \bigvee\{([\![c]\!]^{\#}_A)^n a \mid n \in \mathbb{N}\}$$

Figure 4.4: Abstract semantics for Strat (minus strategy calls).

consider call-by-value procedure calls, so we must extend the proof system appropriately.

The call rule The (call) rule is stated as follows:

$$\frac{strat(x) = s \in Defs \qquad \vdash_A [P[x/y][\overline{z}/\overline{w}]]s[Q]}{\vdash_A [P]strat(y)[Q[y/x][\overline{w}/\overline{z}]]} \text{(call)}$$

The rule simply mirrors the concrete behavior of calls defined in Section 3.2.2. It is obvious, however, that this rule is not sufficient to prove **all** properties of a call, because we allow mutually recursive definitions. We will explain at the end of the chapter why this limitation is not an issue for our goals.

Abstract call We present the abstract semantics of a call by assuming a store abstraction A that is **non-relational**. Then, we can use the same notation defined for concrete stores in the abstract. Therefore, $a[y/x]$ is the store a where x has been replaced by y. An abstract call $[\![strat(y)]\!]^{\#}_A$ will behave like the concrete version but with abstract values instead:

$$call^{\#}(s, x, y, a) = a'[y/x][\overline{w}/\overline{z}] \text{ where } a' = [\![s]\!]^{\#} a[x/y][\overline{z}/\overline{w}] = a'$$

The pointwise definition of the abstraction brings the following properties:

$$a_1 \leq_A a_2 \implies a_1[y/x] \leq_A a_2[y/x] \tag{4.1}$$
$$p \in \mathcal{P}(\Theta) \implies \alpha(p[y/x]) = (\alpha(p))[y/x] \tag{4.2}$$

We omit the proofs, which simply applies the definition of α, $[_/_]$, and $\leq_A$.

4.3 Soundness

It is left to prove that our constructions are correct. We need to demonstrate that the abstract semantics and proof system are **sound**. To avoid

confusion, we will denote with **soundness** the notion from abstract interpretation when dealing with the abstract semantics, while we will talk about **logical soundness** when dealing with the logic itself. The style of the proof is adapted from the original [BGGR21], with the obvious extensions to handle procedure calls.

Induction for procedure calls In the following, we will use structural induction on strategy commands. This works only if we can consider strategy calls as sub-terms. In general this is not true, because the strategy body could be at the same "depth" of the caller (think of recursive calls). To deal with this, we make a minor change to the definition of commands and its semantics. A command s is annotated with a natural number n becoming (s, n). The parameter states that the execution of s will finish with maximum recursive depth of n. This limit ensures termination even with recursive non-terminating calls. Then, the semantics of a call $[\![(strat(y), n)]\!]$ will execute the body s with depth $n - 1$:

$$[\![(strat(y), n)]\!]c = c'[y/x][\overline{w}/\overline{z}] \text{ where } c' = [\![(s, n-1)]\!]c[x/y][\overline{z}/\overline{w}]$$

When we reach depth 0, $\perp$ is returned:

$$[\![(s, 0)]\!]c = \perp = \emptyset$$

Now, the original semantics from Figure 3.6 could be seen as the limit of the annotated version, considering any $n \in \mathbb{N}$:

$$[\![(strat(y))]\!]c = \bigvee_{n \in \mathbb{N}} [\![(strat(y), n)]\!]c$$

By properties of limits, the least fixpoint is preserved. For exposure, in the proofs we will omit the parameter n, and just write $[\![s]\!]$.

4.3.1 Soundness of the abstract semantics

As a first preliminary step, we show that the concrete semantics $[\![_]\!]$ of Strat are **monotone** and **additive**.

Theorem 4.4. *Let $c \in Strat$, and $S_1, S_2 \in \mathcal{P}(\Theta)$. Then*

$$S_1 \subseteq S_2 \implies [\![c]\!]S_1 \subseteq [\![c]\!]S_2 \qquad \text{(Monotonicity)}$$
$$[\![c]\!](S_1 \cup S_2) = [\![c]\!]S_1 \cup [\![c]\!]S_2 \qquad \text{(Additivity)}$$

Proof. **(Monotonicity)**: by induction on the structure of c.

- **(skip)**:

$$[\![\text{skip}]\!]S_1 = S_1 \overset{hp,}{\subseteq} S_2 = [\![\text{skip}]\!]S_2$$

- $\mathbf{e} \in \mathbf{Exp}$: Take $x := s$. Then

$$\begin{aligned} [\![x := s]\!]S_1 &= \\ \{\theta[x \mapsto [\![s]\!]\theta] \mid \theta \in S_1\} &\subseteq \\ \{\theta[x \mapsto [\![s]\!]\theta] \mid \theta \in S_2\} &= \\ [\![x := s]\!]S_2 \end{aligned}$$

  For **test s** and **s <?> p** the reasoning is analogous.

- $\mathbf{c_1}; \mathbf{c_2}$: By induction it holds $[\![c_1]\!]S_1 \subseteq [\![c_1]\!]S_2$ and thus $[\![c_2]\!]([\![c_1]\!]S_1) \subseteq [\![c_2]\!]([\![c_1]\!]S_2)$.

$$[\![c_1; c_2]\!] = [\![c_2]\!][\![c_1]\!]S_1 \subseteq [\![c_2]\!][\![c_1]\!]S_2 = [\![c_1; c_2]\!]S_2$$

- $\mathbf{c_1} \oplus \mathbf{c_2}$: By induction $[\![c_1]\!]S_1 \subseteq [\![c_1]\!]S_2$ and $[\![c_2]\!]S_1 \subseteq [\![c_2]\!]S_2$.

$$[\![c_1 \oplus c_2]\!]S_1 = [\![c_1]\!]S_1 \cup [\![c_2]\!]S_1 \subseteq [\![c_1]\!]S_2 \cup [\![c_2]\!]S_2 = [\![c_1 \oplus c_2]\!]S_2$$

- $\mathbf{c}^*$: We first prove by induction on n the following claim:

$$\forall n \in \mathbb{N} [\![c]\!]^n S_1 \subseteq [\![c]\!]^n S_2$$

  The base case for $n = 0$ is trivial, as $[\![c]\!]^0 S_1 = S_1 \subseteq S_2 = [\![c]\!]^0 S_2$ by hypothesis. For the inductive case:

$$\begin{aligned} [\![c]\!]^{n+1} S_1 = [\![c]\!]([\![c]\!]^n S_1) &\subseteq [\text{by induction}] \\ [\![c]\!]([\![c]\!]^n S_2) &= \\ [\![c]\!]^{n+1} S_2 \end{aligned} \tag{4.3}$$

  It follows that

$$[\![c]\!]^* S_1 = \bigcup\{[\![c]\!]^n S_1 \mid n \in \mathbb{N}\} \overset{4.3}{\subseteq} \bigcup\{[\![c]\!]^n S_2 \mid n \in \mathbb{N}\} = [\![c]\!]^* S_2$$

- $\mathbf{strat(y)}$: Let $S_1 \subseteq S_2$. Then:

$$\begin{aligned} [\![strat(y)]\!]S_1 &= \\ ([\![s]\!]S_1[y/x][\overline{z}/\overline{w}])[y/x][\overline{w}/\overline{z}] &\subseteq [\text{ induction , (3.2)}] \\ ([\![s]\!]S_2[y/x][\overline{z}/\overline{w}])[y/x][\overline{w}/\overline{z}] &= \\ [\![strat(y)]\!]S_2 \end{aligned}$$

(Additivity): by structural induction on c.

- **skip**: trivially

$$[\![\text{skip}]\!](S_1 \cup S_2) = S_1 \cup S_2 = [\![\text{skip}]\!]S_1 \cup [\![\text{skip}]\!]S_2$$

- **x := s**, **test s**, and **s <?> p**: Trivial, as the basic commands are defined pointwise for each store. For exposure, take **x := s**. By definition:

$$\begin{aligned} [\![x := s]\!](S_1 \cup S_2) &= \\ \{\theta[x \mapsto [\![s]\!]\theta] \mid \theta \in S_1 \cup S_2\} &= \\ \{\theta[x \mapsto [\![s]\!]\theta] \mid \theta \in S_1\} \cup \{\theta[x \mapsto [\![s]\!]\theta] \mid \theta \in S_2\} &= \\ [\![x := s]\!]S_1 \cup [\![x := s]\!]S_2 \end{aligned}$$

- $c_1; c_2$: By induction $[\![c_1]\!](S_1 \cup S_2) = [\![c_1]\!]S_1 \cup [\![c_1]\!]S_2$ and $[\![c_2]\!]([\![c_1]\!]S_1 \cup [\![c_1]\!]S_2) = [\![c_2]\!][\![c_1]\!]S_1 \cup [\![c_2]\!][\![c_1]\!]S_2$.

$$\begin{aligned} [\![c_1; c_2]\!](S_1 \cup S_2) &= \\ [\![c_2]\!][\![c_1]\!](S_1 \cup S_2) &= [\text{ induction }] \\ ([\![c_2]\!][\![c_1]\!]S_1) \cup ([\![c_2]\!][\![c_1]\!]S_2) &= \\ [\![c_1; c_2]\!]S_1 \cup [\![c_1; c_2]\!]S_2 \end{aligned}$$

- $c_1 \oplus c_2$:

$$\begin{aligned} [\![c_1 \oplus c_2]\!](S_1 \cup S_2) &= \\ [\![c_1]\!](S_1 \cup S_2) \cup [\![c_2]\!](S_1 \cup S_2) &= [\text{induction on } c_1 \text{and } c_2] \\ [\![c_1]\!]S_1 \cup [\![c_1]\!]S_2 \cup [\![c_2]\!]S_1 \cup [\![c_2]\!]S_2 &= [\text{commutativity of } \cup] \\ ([\![c_1]\!]S_1 \cup [\![c_2]\!]S_1) \cup ([\![c_1]\!]S_2 \cup [\![c_2]\!]S_2) &= \\ ([\![c_1 \oplus c_2]\!]S_1) \cup ([\![c_1 \oplus c_2]\!]S_2) \end{aligned}$$

- c^*: The following claim holds by induction:

$$\forall n \in \mathbb{N} \quad [\![c]\!]^n(S_1 \cup S_2) = [\![c]\!]^n S_1 \cup [\![c]\!]^n S_2$$

For $n = 0$ we have $[\![c]\!]^0(S_1 \cup S_2) = S_1 \cup S_2 = [\![c]\!]^0 S_1 \cup [\![c]\!]^0 S_2$. For the inductive case: $[\![c]\!]^{n+1}(S_1 \cup S_2) = [\![c]\!]^n([\![c]\!](S_1 \cup S_2)) = [\![c]\!]^n([\![c]\!]S_1 \cup [\![c]\!]S_2) = [\![c]\!][\![c]\!]^n S_1 \cup [\![c]\!][\![c]\!]^n S_2 = [\![c]\!]^{n+1}S_1 \cup [\![c]\!]^{n+1}S_2$. Therefore

$$\begin{aligned} [\![c]\!]^*(S_1 \cup S_2) &= \\ \bigcup\{[\![c]\!]^n(S_1 \cup S_2) \mid n \in \mathbb{N}\} &= \\ \bigcup\{[\![c]\!]^n S_1 \cup [\![c]\!]^n S_2 \mid n \in \mathbb{N}\} &= [\cup \text{ commutative}] \\ \bigcup\{[\![c]\!]^n S_1 \mid n \in \mathbb{N}\} \cup \bigcup\{[\![c]\!]^n S_2 \mid n \in \mathbb{N}\} &= \\ [\![c]\!]^* S_1 \cup [\![c]\!]^* S_2 \end{aligned}$$

- **strat(y)**: Let S_1, S_2 two stores and $strat(x) = s \in Defs$. Then:

$$\begin{aligned}
[\![strat(y)]\!](S_1 \cup S_2) &= \\
([\![s]\!](S_1 \cup S_2)[x/y][\overline{z}/\overline{w}])[y/x][\overline{w}/\overline{z}] &= [\text{ induction }, (3.1)] \\
([\![s]\!]S_1[x/y][\overline{z}/\overline{w}])[y/x][\overline{w}/\overline{z}] \cup ([\![s]\!]S_2[x/y][\overline{z}/\overline{w}])[y/x][\overline{w}/\overline{z}] &= \\
[\![strat(y)]\!]S_1 \cup [\![strat(y)]\!]S_2
\end{aligned}$$

□

We now prove **monotonicity** and **soundness** of the abstract semantics.

Theorem 4.5. *Let $c \in Strat$, $A \in Abs_{\alpha,\gamma}(\mathcal{P}(\Theta)), p \in \mathcal{P}(\Theta), a_1, a_2 \in A$. Then*

$$a_1 \leq_A a_2 \implies [\![c]\!]^{\#}_A a_1 \leq_A [\![c]\!]^{\#}_A a_2 \qquad \text{(monotonicity)}$$

$$\alpha \circ [\![c]\!]p \leq_A [\![c]\!]^{\#}_A \circ \alpha(p) \qquad \text{(soundness)}$$

Proof. **(Monotonicity)**: by structural induction on c.

- $\mathbf{e} \in \mathbf{Exp}$:

$$\begin{aligned}
[\![c]\!]^{\#}_A a_1 &= \\
\alpha([\![c]\!]\gamma(a_1)) &\leq [\gamma, \alpha, [\![_]\!] \text{ monotone}] \\
\alpha([\![c]\!]\gamma(a_2)) &= \\
[\![c]\!]^{\#}_A a_2
\end{aligned}$$

- $c_1; c_2$: By induction

$$\begin{aligned}
[\![c_1]\!]^{\#}_A a_1 &\leq_A [\![c_1]\!]^{\#}_A a_2 \\
[\![c_2]\!]^{\#}_A [\![c_1]\!]^{\#}_A a_1 &\leq_A [\![c_2]\!]^{\#}_A [\![c_1]\!]^{\#}_A a_2
\end{aligned}$$

  Then

$$[\![c_1; c_2]\!]^{\#}_A a_1 = [\![c_2]\!]^{\#}_A [\![c_1]\!]^{\#}_A a_1 \leq_A [\![c_2]\!]^{\#}_A [\![c_1]\!]^{\#}_A a_2 = [\![c_1; c_2]\!]^{\#}_A a_2$$

- $c_1 \oplus c_2$: By induction

$$\begin{aligned}
[\![c_1]\!]^{\#}_A a_1 &\leq_A [\![c_1]\!]^{\#}_A a_2 \\
[\![c_2]\!]^{\#}_A a_1 &\leq_A [\![c_2]\!]^{\#}_A a_2
\end{aligned}$$

  Then

$$[\![c_1 \oplus c_2]\!]^{\#}_A a_1 = [\![c_1]\!]^{\#}_A a_1 \vee [\![c_2]\!]^{\#}_A a_1 \leq_A [\![c_1]\!]^{\#}_A a_2 \vee [\![c_2]\!]^{\#}_A a_2 = [\![c_1 \oplus c_2]\!]^{\#}_A a_2$$

- c^*: By induction on n we prove:

$$\forall n \in \mathbb{N}(\llbracket c \rrbracket_A^{\#})^n a_1 \leq_A (\llbracket c \rrbracket_A^{\#})^n a_2 \tag{4.4}$$

For $n = 0$ we have $(\llbracket c \rrbracket_A^{\#})^0 a_1 = a_1 \leq_A a_2 = (\llbracket c \rrbracket_A^{\#})^n a_2$. For the inductive case:

$$(\llbracket c \rrbracket_A^{\#})^{n+1} a_1 = \llbracket c \rrbracket_A^{\#} (\llbracket c \rrbracket_A^{\#})^n a_1 \leq_A \llbracket c \rrbracket_A^{\#} (\llbracket c \rrbracket_A^{\#})^n a_2 = (\llbracket c \rrbracket_A^{\#})^{n+1} a_2$$

Then

$$(\llbracket c \rrbracket_A^{\#})^* a_1 = \bigvee \{(\llbracket c \rrbracket_A^{\#})^n a_1 \mid n \in \mathbb{N}\} \overset{4.4}{\leq_A} \bigvee \{(\llbracket c \rrbracket_A^{\#})^n a_2 \mid n \in \mathbb{N}\} = (\llbracket c \rrbracket_A^{\#})^* a_2$$

- **strat(y)**: By (4.1) and induction

$$\llbracket s \rrbracket_A^{\#} a_1[x/y][\overline{z}/\overline{w}] \leq_A \llbracket s \rrbracket_A^{\#} a_2[x/y][\overline{z}/\overline{w}] \tag{4.5}$$

Then:

$$\begin{aligned} \llbracket strat(y) \rrbracket_A^{\#} a_1 &= \\ (\llbracket s \rrbracket_A^{\#}[x/y][\overline{z}/\overline{w}] a_1)[y/x][\overline{w}/\overline{z}] &\leq [(4.5), (4.1)] \\ (\llbracket s \rrbracket_A^{\#}[x/y][\overline{z}/\overline{w}] a_2)[y/x][\overline{w}/\overline{z}] &= \\ \llbracket strat(y) \rrbracket_A^{\#} a_2 \end{aligned}$$

(Soundness): by structural induction on c.

- $\mathbf{e} \in \mathbf{Exp}$: Recall from the definition of Galois connections that

$$p \leq_C \gamma(\alpha(p)) \tag{4.6}$$

Then

$$\begin{aligned} \alpha(\llbracket \mathrm{e} \rrbracket p) &\leq_a [(4.6), \alpha \text{ and } \llbracket _ \rrbracket \text{ monotone}] \\ \alpha(\llbracket \mathrm{e} \rrbracket \gamma(\alpha(p))) &= \\ \llbracket \mathrm{e} \rrbracket_A^{\#} \alpha(p) \end{aligned}$$

- $c_1; c_2$:

$$\begin{aligned} \alpha(\llbracket c_1; c_2 \rrbracket p) = \alpha(\llbracket c_2 \rrbracket \llbracket c_1 \rrbracket p) &\leq_A [\text{induction on } c_2] \\ \llbracket c_2 \rrbracket_A^{\#} \alpha(\llbracket c_1 \rrbracket p) &\leq_A [\text{induction on } c_1] \\ \llbracket c_2 \rrbracket_A^{\#} \llbracket c_1 \rrbracket_A^{\#} \alpha(p) &= \\ \llbracket c_1; c_2 \rrbracket_A^{\#} \alpha(p) \end{aligned}$$

- $c_1 \oplus c_2$:

$$\begin{aligned}
\alpha([\![c_1 \oplus c_2]\!]p) &= \\
\alpha([\![c_1]\!]p \cup [\![c_2]\!]p) &= [\text{additivity of } \alpha] \\
\alpha([\![c_1]\!]p) \cup \alpha([\![c_2]\!]p) &\leq_A [\text{induction}] \\
[\![c_1]\!]^{\#}_A \alpha(p) \cup [\![c_2]\!]^{\#}_A \alpha(p) &= \\
[\![c_1 \oplus c_2]\!]^{\#}_A \alpha(p)
\end{aligned}$$

- c^*: We prove the auxiliary lemma:

$$\forall n \in \mathbb{N}. \quad \alpha([\![c]\!]^n p) \leq_A ([\![c]\!]^{\#}_A)^n \alpha(p) \tag{4.7}$$

For $n = 0$ we have $\alpha([\![c]\!]^0 p) = \alpha(p) = ([\![c]\!]^{\#}_A)^0 \alpha(p)$. For the inductive case: $\alpha([\![c]\!]^{n+1} p) = \alpha([\![c]\!]^n [\![c]\!]p) \overset{4.7}{\leq_A} ([\![c]\!]^{\#}_A)^n \alpha([\![c]\!]p) \leq_A ([\![c]\!]^{\#}_A)^n [\![c]\!]^{\#}_A \alpha(p) = ([\![c]\!]^{\#}_A)^{n+1} \alpha(p)$. It follows that

$$\begin{aligned}
\alpha([\![c^*]\!]p) &= \\
\alpha(\bigvee\{[\![c]\!]^n p) \mid n \in \mathbb{N}\} &= [\text{additivity of } \alpha] \\
\bigvee\{\alpha([\![c]\!]^n p) \mid \ n \in \mathbb{N}\}) &\leq_A [(4.7)] \\
\bigvee\{([\![c]\!]^{\#}_A)^n \alpha(p) \mid n \in \mathbb{N}\} &= \\
([\![c]\!]^{\#}_A)^* \alpha(p)
\end{aligned}$$

- **strat(y)**: Let $strat(x) = s \in Defs$. Then:

$$\begin{aligned}
\alpha([\![strat(y)]\!]p) &= \\
\alpha(([\![s]\!]p[x/y][\overline{z}/\overline{w}])[y/x][\overline{w}/\overline{z}]) &= [(4.2)] \\
(\alpha([\![s]\!]p[x/y][\overline{z}/\overline{w}]))[y/x][\overline{w}/\overline{z}] &\leq [\text{induction }, (4.1)] \\
([\![s]\!]^{\#}_A \alpha(p[x/y][\overline{z}/\overline{w}]))[y/x][\overline{w}/\overline{z}] &= \\
[\![strat(y)]\!]^{\#}_A \alpha(p)
\end{aligned}$$

□

4.3.2 Logical soundness

Finally, we prove **logical soundness** of the proof system.

Theorem 4.6. *(Logical soundness) Let* $\vdash_A [P]c[Q]$. *Then*

$$Q \leq [\![c]\!]P \tag{LS1}$$

$$[\![c]\!]^{\#}_A \alpha(P) = \alpha(Q) = \alpha([\![c]\!]P) \tag{LS2}$$

Proof. The right equality of (LS2) can be derived from the left one, (LS1), and soundness:

$$Q \leq_C [\![c]\!]P \overset{\text{monot. of } \alpha}{\Longrightarrow} \alpha(Q) \leq_A \alpha([\![c]\!]P)$$

$$\alpha([\![c]\!]P) \overset{sound.}{\leq_A} [\![c]\!]^{\#}_A \alpha(P) = \alpha(Q)$$

By anti-symmetry of $\leq_A$, $\alpha(Q) = \alpha([\![c]\!]P)$. We are left to prove (LS1) and $[\![c]\!]^{\#}_A \alpha(P) = \alpha(Q)$.

The proof is by induction on the derivation tree.

- (**transfer**). (LS1) is trivial, as $\vdash [P]e[[\![e]\!]P]$ and $[\![e]\!]P \leq [\![e]\!]P$. For (LS2), we use the local completeness hypothesis:

$$\begin{aligned} [\![e]\!]^{\#}_A \alpha(P) &= \\ \alpha([\![e]\!]\gamma(\alpha(P))) &= [\mathbb{C}^A_P([\![e]\!]P)] \\ \alpha([\![e]\!]P) &= [\text{(LS2)}] \\ \alpha(Q) & \end{aligned}$$

- (**relax**) For (LS1), by hypothesis $Q \leq_C Q'$ and $P' \leq_C P$. Then, it follows

$$Q \leq_C Q' \leq_C [\![c]\!]P' \leq_C [\![c]\!]P$$

  by using the inductive hypothesis for $Q' \leq_C [\![c]\!]P'$, and monotonicity of $[\![_]\!]$ for $[\![c]\!]P' \leq_C [\![c]\!]P$. For (LS2), we want to prove $[\![c]\!]^{\#}_A \alpha(P) = \alpha(Q)$. Now, by induction it holds $[\![c]\!]^{\#}_A \alpha(P') = \alpha(Q')$. Then, to prove (LS2), it is enough to show that $\alpha(P') = \alpha(P)$ and $\alpha(Q') = \alpha(Q)$. By hypothesis $P' \leq_C P \leq_C A(P')$. Recall that $A = \gamma \circ \alpha$. We reapply it, and by monotonicity of α and γ, we get $A(P') \leq_C A(P) \leq_C A(A(P'))$. Now, in a Galois insertion, $A \circ A = A$. Thus, $A(A(P')) = A(P')$. The previous inequality becomes $A(P') \leq_C A(P) \leq_C A(P')$, which proves $A(P) = A(P')$ by anti-symmetry of $\leq_C$. Following an analogous reasoning, we get $A(Q) = A(Q')$, thus proving our claim.

- (**seq**) By induction, $Q \leq_C [\![c_2]\!]R$ and $R \leq_C [\![c_1]\!]P$. Then, for (LS1):

$$\begin{aligned} Q \leq_C [\![c_2]\!]R \leq_C & [[\![_]\!] \text{ monotone }] \\ [\![c_2]\!][\![c_1]\!]P = & \\ [\![c_1; c_2]\!]P & \end{aligned}$$

For (LS2), we use the inductive hypothesis:

$$[\![c_1; c_2]\!]^{\#}_A \alpha(P) = [\![c_2]\!]^{\#}_A [\![c_1]\!]^{\#}_A \alpha(P) = [\![c_2]\!]^{\#}_A \alpha(R) = \alpha(Q)$$

- (**join**) For (LS1), by induction $Q_1 \leq_C [\![c_1]\!]P$ and $Q_2 \leq_C [\![c_2]\!]P$. Then:

$$\begin{aligned} Q_1 \vee_C Q_2 \leq_C & \\ [\![c_1]\!]P \vee_C [\![c_2]\!]P = & \\ [\![c_1 \oplus c_2]\!]P & \end{aligned}$$

For (LS2), we use the inductive hypothesis and additivity of α:

$$\begin{aligned} \alpha(Q_1 \vee_C Q_2) = & [\text{additivity of } \alpha] \\ \alpha(Q_1) \vee_C \alpha(Q_2) = & [\text{induction}] \\ [\![c_1]\!]^{\#}_A \alpha(P) \vee_A [\![c_2]\!]^{\#}_A \alpha(P) = & \\ [\![c_1 \oplus c_2]\!]^{\#}_A \alpha(P) & \end{aligned}$$

- (**iterate**) For (LS1), by induction $Q \leq_C [\![c]\!]P$. Therefore:

$$P \vee Q \leq_C P \vee [\![c]\!]P = [\![c]\!]^0 P \vee [\![c]\!]^1 P \leq_C \bigvee \{[\![c]\!]^n P \mid n \in \mathbb{N}\} = [\![c]\!]^* P$$

For (LS2), as a first step we prove by the following lemma for a generic $e \in C$.

$$[\![c]\!]^{\#}_A \alpha(e) \leq_A \alpha(e) \implies [\![c^*]\!]^{\#}_A \alpha(e) = \alpha(e) \tag{4.8}$$

Assume $[\![c]\!]^{\#}_A \alpha(e) \leq_A \alpha(e)$. Then, we show by induction

$$\forall n \geq 1. [\![c^n]\!]^{\#}_A \alpha(e) \leq_A \alpha(e)$$

The base case $n = 1$ is our assumption. For the inductive step:

$$[\![c^{n+1}]\!]^{\#}_A \alpha(e) = [\![c]\!]^{\#}_A [\![c^n]\!]^{\#}_A \alpha(e) = [\![c]\!]^{\#}_A \alpha(e) = \alpha(e)$$

Thus,

$$\begin{aligned}
&[\![c^*]\!]^{\#}_A \alpha(e) = \\
&\bigvee\{[\![c^n]\!]^{\#}_A \alpha(e) \mid n \in \mathbb{N}\} = [\text{ induction and} [\![c^0]\!]^{\#}_A \alpha(e) = \alpha(e)] \\
&\alpha(e)
\end{aligned}$$

Now we go back to the proof for (iterate). By induction and hypothesis $Q \leq_C A(P)$, we have $[\![c]\!]^{\#}_A \alpha(P) = \alpha(Q) \leq_A \alpha(A(P)) = \alpha(P)$. Therefore, we can apply (4.8) and obtain:

$$\begin{aligned}
[\![c^*]\!]^{\#}_A \alpha(P) &= [(4.8)] \\
\alpha(P) &= [Q \leq A(P)] \\
\alpha(P) \vee \alpha(Q) &= [\alpha \text{ additive }] \\
\alpha(P \vee Q)
\end{aligned}$$

- (**rec**) For (LS1), by induction it holds $R \leq_C [\![c]\!]P$ and $Q \leq_C [\![c^*]\!](P \vee R)$. From the first hypothesis, we show $[\![c^*]\!](P \vee R) = [\![c^*]\!]P$:

$$\begin{aligned}
[\![c^*]\!]P &\leq [[\![_]\!] \text{ monotone }] \\
[\![c^*]\!](P \vee R) &\leq [R \leq [\![c]\!]P] \\
[\![c^*]\!](P \vee [\![c]\!]P) &= [[\![c]\!]P \leq [\![c^*]\!]P] \\
[\![c^*]\!](P \vee [\![c^*]\!]P) &= [P \leq [\![c^*]\!]P] \\
[\![c^*]\!]([\![c^*]\!]P) &= [\text{ idempotency}] \\
[\![c^*]\!]P
\end{aligned} \tag{4.9}$$

It follows that:

$$\begin{aligned}
Q &\leq_C [\text{ induction }] \\
[\![c^*]\!](P \vee R) &= [(4.9)] \\
[\![c^*]\!](P)
\end{aligned}$$

For (LS2), by induction $\alpha(R) = [\![c]\!]^{\#}_A \alpha(P)$ and $\alpha(Q) = [\![c^*]\!]^{\#}_A \alpha(P \vee R)$. Then

$$\begin{aligned}
\alpha(Q) &\leq_A [(\text{LS1}), \alpha \text{ monotone }] \\
\alpha([\![c^*]\!](P)) &\leq_A [\text{ soundness of } [\![_]\!]^{\#}_A] \\
[\![c^*]\!]^{\#}_A \alpha(P) &\leq_A [\text{ monotonicity }] \\
[\![c^*]\!]^{\#}_A \alpha(P \vee R) &= [\text{ induction }] \\
\alpha(Q)
\end{aligned}$$

It follows $[\![c^*]\!]^{\#}_A \alpha(P) = \alpha(Q)$.

- (**call**) For (LS1), by induction $Q \leq [\![s]\!]P[x/y][\overline{z}/\overline{w}]$. Therefore:

$$\begin{aligned} Q[y/x][\overline{w}/\overline{z}] &\leq_C [\text{ induction }, (4.1)] \\ ([\![s]\!]P[x/y][\overline{z}/\overline{w}])[y/x][\overline{w}/\overline{z}] &= \\ [\![strat(y)]\!]P \end{aligned}$$

For (LS2), by induction $\alpha(Q) = [\![s]\!]^{\#}_A \alpha(P[x/y][\overline{z}/\overline{w}])$. Then:

$$\begin{aligned} \alpha(Q[y/x][\overline{z}/\overline{w}]) &= [(4.2)] \\ (\alpha(Q))[y/x][\overline{z}/\overline{w}] &= [\text{ induction }] \\ ([\![s]\!]^{\#}_A \alpha(P[x/y][\overline{z}/\overline{w}]))[y/x][\overline{z}/\overline{w}] &= \\ [\![s]\!]^{\#}_A \alpha(P) \end{aligned}$$

□

As a consequence, if a correctness specification $Spec$ is expressible in A, i.e., if $Spec = \gamma(a)$ for some $a \in A$, then any provable triple $\vdash_A [P]c[Q]$ allows us to use Q to reason about either the *correctness* or *incorrectness* of c for the pre-condition P, as stated by the following result:

Corollary 4.7. ***(Precision)*** *For all $A \in Abs_{\alpha,\gamma}(C)$, $c \in Strat$, $P, Q \in C$, if $\vdash_A [P]c[Q]$, then:*

$$\forall a \in A. [\![c]\!]P \leq \gamma(a) \textit{ iff } Q \leq \gamma(a)$$

4.4 On logical completeness

When we presented the rule (call), we mentioned how it was not enough to prove all properties of function calls. For any non-terminating, mutually recursive procedure, the proof never terminates as well. The problem of defining an axiomatic system capable of dealing with these situations is well studied in literature. Typically, one defines a proof context Γ that stores all true premises that can be used when proving a triple. Then, the solution for recursive calls is to add the conclusion in the context to prove the premise. In our system, this would mean to add two rules similar to these:

$$\frac{strat(x) = s \in Defs \qquad [P]strat(y)[Q[y/x][\overline{w}/\overline{z}]], \Gamma \vdash_A [P[x/y][\overline{z}/\overline{w}]]s[Q]}{\vdash_A [P]strat(y)[Q[y/x][\overline{w}/\overline{z}]]} \text{(callR)}$$

$$\frac{t \in \Gamma}{\vdash_A t} \text{(asm)}$$

The rule (asm) allows use to use any true facts in the context Γ. The problem is that, in LCL, this is not enough to recover completeness in the logical sense. In the original paper [BGGR21](Theorem V.11-12), it was shown that to recover completeness, one need to:

1. add a sound, infinitary rule (limit) for Kleene star:

$$\frac{\forall n \in \mathbb{N}.\ \vdash_A [P_n]c[P_{n+1}]}{\vdash_A [P_0]c^*[\bigvee_{i\in\mathbb{N}} P_i]} \text{(limit)}$$

2. assume that all basic expressions e $\in Exp$ are globally complete in A.

Not only that, the authors also demonstrated how, for a Turing complete language, the proof system is **intrinsically incomplete** for any non-trivial abstraction, meaning that there always is a non-provable triple. For these reasons, we decided against adding rules (callR) and (asm) to our system, and assume to be dealing with terminating calls.

Chapter 5

Analysis

In this chapter, we present two test programs written in **Strat** and their analysis using the LCL framework. Both examples are meant to represent relevant properties of **Strat**, provable using LCL (and to illustrate its limitations). For each program, we show a test execution and the full derivation in the proof system. During each test, a set of preconditions P is assumed.

Term abstraction In the examples, we will utilize the following procedure to define the abstraction of terms over a signature Σ.

First, a **finite** set $Shapes$ is established. An element of this set describes a certain subset of T_Σ, characterized by a specific structure that is relevant to our analysis. For example, the set $\{f, g, h\}$ could describe terms with root symbol f, g or h. We assume that the sets of terms defined by each shape are **disjoint**.

The mapping of a term t to its abstract shape is determined by a function $\eta : T_\Sigma \to Shapes$. Analogously, the function $\rho : Shapes \to \mathcal{P}(T_\Sigma)$ gives, for each abstract shape a, the set of concrete terms described by it.

Now, for the concrete domain $\mathcal{P}(T_\Sigma)$, we take $\mathcal{P}(Shapes)$ as its abstraction. Each set of terms is abstracted by a subset of $Shapes$, standing in for the classes of terms included. The abstraction and concretization functions γ and α make use of η and ρ as follows:

$$P \in \mathcal{P}(T_\Sigma) \quad A \in \mathcal{P}(Shapes)$$

$$\alpha(P) = \{\eta(p) \mid p \in P\}$$

$$\gamma(A) = \bigcup_{a \in A} \rho(a)$$

As an example, consider again the set $Shapes = \{f, g, h\}$ from before. The

functions η and ρ distinguish terms on their root symbols:

$$\eta(t) = \begin{cases} f & \text{if } t = f(t_1) \quad t_1 \in T_\Sigma \\ g & \text{if } t = g(t_1, t_2) \quad t_1, t_2 \in T_\Sigma \\ h & \text{if } t = h(t_1, t_2) \quad t_1, t_2 \in T_\Sigma \end{cases}$$

$$\rho(a) = \begin{cases} \{f(t_1) \mid t_1 \in T_\Sigma\} & \text{if } a = f \\ \{g(t_1, t_2) \mid t_1, t_2 \in T_\Sigma\} & \text{if } a = g \\ \{h(t_1, t_2) \mid t_1, t_2 \in T_\Sigma\} & \text{if } a = h \end{cases}$$

A concrete set $T = \{f(a), g(b, c), g(a, d)\}$ would be abstracted by $\{f, g\}$:

$$a = \alpha(T) = \{f, g\}$$

Conversely, the abstract point a is concretized to the union of terms with root symbol f, and those with root symbol g:

$$\gamma(a) = \{f(t) \mid t \in T_\Sigma\} \cup \{g(t_1, t_2) \mid t_1, t_2 \in T_\Sigma\}$$

5.1 Term shape

For the initial example, we take the following TRS and strategy program:

$$\Sigma = \{c, f, g, h\}$$

$$fg : f(x) \to g(x), \qquad hf : h(f(x), y) \to f(y) \qquad hg : h(g(x), y) \to g(y)$$

```
hrules = hf <+ hg

red(x) =
    if ?h(a,b)@x then
        x = try(hrules)
    else
        x := !g(x)

main(y) =
    red(y);
    if ?f(x)@y then
        x @= fg;
    else
        skip
```

We would like to predict the shape of x after executing main(x). As a first step, the abstract domain is built as explained in the introduction. Figure 5.1 exposes the set $Shapes$ and the functions η and ρ. We divide T_Σ based on the shape of terms up until the direct children of the root symbol, to follow the patterns defined in the rules of the TRS.

$$Shapes = \{c, hc, hg, hf, f, g, fail\}$$

$$\eta(t) = \begin{cases} fail & \text{if } t = KO \\ c & \text{if } t = c \\ f & \text{if } t = f(t_1) \quad t_1 \in T_\Sigma \\ g & \text{if } t = g(t_1) \quad t_1 \in T_\Sigma \\ hf & \text{if } t = h(f(t_1), t_2) \quad t_1, t_2 \in T_\Sigma \\ hg & \text{if } t = h(g(t_1), t_2) \quad t_1, t_2 \in T_\Sigma \\ ho & \text{o.w.} \end{cases}$$

$$\rho(a) = \begin{cases} \{fail\} & \text{if } a = KO \\ \{c\} & \text{if } a = c \\ \{f(t_1) \mid t_1 \in T_\Sigma\} & \text{if } a = f \\ \{g(t_1) \mid t_1 \in T_\Sigma\} & \text{if } a = g \\ \{h(f(t_1), t_2) \mid t_1, t_2 \in T_\Sigma\} & \text{if } a = hf \\ \{h(g(t_1), t_2) \mid t_1, t_2 \in T_\Sigma\} & \text{if } a = hg \\ \{h(t_1, t_2) \mid t_1 = c \vee t_1 = h(t_2, t_3).\ t_1, t_2, t_3 \in T_\Sigma\} & \text{if } a = ho \end{cases}$$

Figure 5.1: Abstract domain for the first example.

To begin the analysis, we desugar the if-else statements:

```
red(x) =
    (test ?h(a,b)@x; x = try(hrules))
    ⊕
    (test not(?h(a,b)@x); x = !g(x))

main(y) =
    red(y);
    (test ?f(x)@y; x @= fg)
    ⊕
    (test not(?f(x)@y); skip)
```

For brevity, we will use the following abbreviations:

$r_1 \triangleq$ test ?h(a,b)@x $\quad r_2 \triangleq x = try(hrules)$

$r_3 \triangleq$ test not(?h(a,b)@x) $\quad r_4 \triangleq$ x := !g(x)

$r \triangleq (r_1; r_2) \oplus (r_3; r_4)$

$b_1 \triangleq$ test ?f(x)@y $\quad b_2 \triangleq x@ = fg$

$b_3 \triangleq$ test not(?f(x)@y) $\quad b_4 \triangleq skip$

$b \triangleq (b_1; b_2) \oplus (b_3; b_4)$

Because the program works on a single variable y, $\mathcal{P}(T_\Sigma)$ is used to represent set of stores. A set $T \in \mathcal{P}(T_\Sigma)$ stands for the set $\{\theta \in S \mid \theta(y) \in T\}$, where $S \in \mathcal{P}(\Theta)$.

As a consequence, an abstraction $a \in Abs(T_\Sigma)$ represents an abstract store $Abs(\mathcal{P}(\Theta))$. For instance, $\{f(c), h(g(c), c)\}$ stands for the expression $y = f(c) \vee y = h(g(c), c)$.

Consider now the specification $Spec = y \in \{g(x) \mid x \in T_\Sigma\}$. Figure 5.2 details the proof for the statement

$$\vdash_A [P]main(y)[Q] \tag{5.1}$$

and the conditions

$$P = \{f(c), h(f(c), g(c))\} = P_x$$

$$P1_x = \{h(f(c), g(c))\} \qquad P2_x = \{f(g(c))\}$$
$$P3_x = \{f(c)\} \qquad P4_x = g(f(c))$$

$$Q_x = P_1 = \{g(f(c)), f(g(c))\} \qquad P_2 = \{f(g(c))\}$$
$$P_3 = \{g(g(c))\} \qquad P4 = \{g(f(c))\}$$

$$Q = P3 \cup P4 = \{g(f(c)), g(g(c))\}$$

The subscript x is used to differentiate between the conditions used in main, which refers to the value of y, from the conditions inside the call **red(y)**, where instead we keep track of the variable x. During a call, we only track the relevant variables and ignore those created by the substitution $[\overline{z}/\overline{w}]$ at the start of the call.

Note how each basic transfer expression enforces a proof obligation requiring local completeness (**(test)** and **(assign)** rules). It is easy to see that

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{P_x}(r_1)}{\vdash_A [P_x]r_1[P1_x]}\,\text{(test)} \quad \dfrac{\mathbb{C}^A_{P1_x}(r_2)}{\vdash_A [P1_x]r_2[P2_x]}\,\text{(ass.)}}{\vdash_A [P_x]r_1;r_2[P2_x]}\,\text{(seq)} \quad \dfrac{\dfrac{\mathbb{C}^A_{P_x}(r_3)}{\vdash_A [P_x]r_3[P3_x]}\,\text{(test)} \quad \dfrac{\mathbb{C}^A_{P3_x}(r_4)}{\vdash_A [P3_x]r_4[P4_x]}\,\text{(ass.)}}{\vdash_A [P_x]r_3;r_4[P4_x]}}{(*)}\,\text{(join)}$$

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{P1}(b_1)}{\vdash_A [P1]b_1[P2]}\,\text{(test)} \quad \dfrac{\mathbb{C}^A_{P2}(b_2)}{\vdash_A [P2]b_2[P3]}\,\text{(ass.)}}{\vdash_A [P1]b_1;b_2[P3]}\,\text{(seq)} \quad \dfrac{\dfrac{\mathbb{C}^A_{P1}(b_3)}{\vdash_A [P1]b_3[P4]}\,\text{(test)} \quad \dfrac{}{\vdash_A [P4]b_4[P4]}\,\text{(skip)}}{\vdash_A [P1]b_3;b_4[P4]}}{(**)}\,\text{(join)}$$

$$\dfrac{\dfrac{\dfrac{(*)}{\vdash_A [P_x]r[Q_x]}\,\text{(join)}}{\vdash_A [P]red(y)[P1]}\,\text{(call)} \quad \dfrac{(**)}{\vdash_A [P1]b[Q]}\,\text{(join)}}{\vdash_A [P]main(y)[Q]}\,\text{(seq)}$$

Figure 5.2: Derivation of (5.1).

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{P'_x}(r_1)}{\vdash_A [P'_x]r_1[P1'_x]}\,\text{(test)} \quad \dfrac{\mathbb{C}^A_{P1'_x}(r_2)}{\vdash_A [P1'_x]r_2[P1'_x]}\,\text{(ass.)}}{\vdash_A [P_x]r_1;r_2[P1'_x]}\,\text{(seq)} \quad \dfrac{\dfrac{\mathbb{C}^A_{P'_x}(r_3)}{\vdash_A [P'_x]r_3[P2'_x]}\,\text{(test)} \quad \dfrac{\mathbb{C}^A_{P2'_x}(r_4)}{\vdash_A [P2'_x]r_4[P3'_x]}\,\text{(ass.)}}{\vdash_A [P'_x]r_3;r_4[P3'_x]}}{(*)}\,\text{(join)}$$

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{Q'}(b_1)}{\vdash_A [Q']b_1[\emptyset]}\,\text{(test)} \quad \dfrac{\mathbb{C}^A_{\emptyset}(b_2)}{\vdash_A [\emptyset]b_2[\emptyset]}\,\text{(ass.)}}{\vdash_A [Q']b_1;b_2[\emptyset]}\,\text{(seq)} \quad \dfrac{\dfrac{\mathbb{C}^A_{Q'}(b_3)}{\vdash_A [Q']b_3[Q']}\,\text{(test)} \quad \dfrac{}{\vdash_A [Q']b_4[Q']}\,\text{(skip)}}{\vdash_A [Q']b_3;b_4[Q']}}{(**)}\,\text{(join)}$$

$$\dfrac{P' \subseteq P_{fail} \subseteq A(P') = \{ho, f\} \quad \dfrac{\dfrac{\dfrac{(*)}{\vdash_A [P'_x]r[Q'_x]}\,\text{(join)}}{\vdash_A [P']red(y)[Q']}\,\text{(call)} \quad \dfrac{(**)}{\vdash_A [Q']b[Q']}\,\text{(join)}}{\vdash_A [P']main(y)[Q']}\,\text{(seq)} \quad Q' \subseteq Q' \subseteq A(Q')}{\vdash_A [P_{fail}]main(y)[Q']}\,\text{(relax)}$$

Figure 5.3: Derivation of (5.2).

the final post-condition $Q = \{g(f(c)), g(g(c))\}$ satisfies $Spec$. More importantly, $Spec$ is **expressible** in A ($Spec = \gamma(g)$), so we can conclude that in **any** triple $\vdash_A [P]main(y)[Q']$, $Spec$ is satisfied.

Similarly, because $Spec$ corresponds to a point in the abstract domain, any alert raised is a true alert. Take the following conditions:

$$P_{fail} = \{h(c, f(g(c))), f(f(c)), f(h(g(c), c))\} \quad P' = \{h(c, f(g(c))), f(f(c))\} = P'_x$$
$$P1'_x = \{h(c, f(g(c)))\} \quad P2'_x = \{f(f(c))\}$$
$$P3'_x = \{g(f(f(c)))\} \quad Q' = \{h(c, f(g(c))), g(f(f(c)))\} = Q'_x$$

The derivation

$$\vdash_A [P_{fail}]main(y)[Q'] \tag{5.2}$$

is shown in Figure 5.3. We exploit **relax** to ignore the value $f(h(g(c), c))$ and obtain P'. This is valid, since $\alpha(\{h(c, f(g(c))), f(f(c))\}) = \{ho, f\}$, which already denotes a superset of P_{fail}. The final post-condition $Q' = \{h(c, f(g(c))), g(f(f(c)))\}$ **does not** satisfy $Spec$, and the proof exposes the true alert $h(c, f(g(c)))$.

5.2 Even Numbers

Consider the first strategy program from Example 3.1 once again:

```
evenN = evenZero <+ evenOne <+ evenSucc
```

norm(evenN, n) = (test evenN@n; n @=evenN)*;test not(evenN)@n;

The target property that we want to hold is that, for any even number n, the program rewrites $even(n)$ to $true$. To improve readability, and avoid cluttering the proofs, we will refer to algebraic numbers **zero and succ(n)** with the $succ^n$ notation, which is defined inductively:

$$succ^n = \begin{cases} zero & \text{if n = 0} \\ succ(succ^{n-1}) & \text{if } n > 0 \end{cases}$$

The sets of even and odd numbers are defined as follows:

$$Even = \{succ^{2n} \mid n \in \mathbb{N}\}$$

$$Odd = \{succ^{2n+1} \mid n \in \mathbb{N}\}$$

Abstract domain

We first delineate a proper abstraction A. The set $Shapes$ and the functions η and ρ are presented in Figure 5.4. The main partition is between $\mathbf{even(n)}$ terms with n an even number and $\mathbf{even(m)}$ terms with m an odd number. The former are represented by the shape symbol e, and the latter are denoted by o. The terms $true$ and $false$ are also expressed in A by the tt and ff shapes, while $fail$ represents the KO term as expected. Every other term is mapped to a instead.

$$Shapes = \{fail, tt, ff, e, o, a\}$$

$$\eta(t) = \begin{cases} tt & \text{if } t = true \\ ff & \text{if } t = false \\ e & \text{if } t = even(n), n \in Even \\ o & \text{if } t = even(n), n \in Odd \\ fail & \text{if } t = KO \\ a & \text{o.w.} \end{cases}$$

$$\rho(t) = \begin{cases} \{true\} & \text{if } t = tt \\ \{false\} & \text{if } t = ff \\ \{KO\} & \text{if } t = fail \\ \{even(n) \mid n \in Even\} & \text{if } t = e \\ \{even(n) \mid n \in Odd\} & \text{if } t = o \\ \{even(n) \mid n \notin Even \cup Odd\} \cup \{zero\} \cup \{succ(t) \mid t \in T_\Sigma\} & \text{if } t = a \end{cases}$$

Figure 5.4: Abstraction for the even program.

Analysis

We let

$c \triangleq$ norm(evenN,n) $=$ (test evenN@n; n @=evenN)*;test not(evenN)@n;

Our claim is that, for the precondition $P = \{even(n) \mid n \in Even\}$, c satisfies the correctness specification $Spec \triangleq n = true$. The triple $\vdash_A [P]\ c\ [\{true\}]$ can be derived as shown in Figure 5.5, where for brevity we let:

$c_1 \triangleq$ (test $evenN@n; n@ = evenN)^*$ $\quad$ $c_2 \triangleq$ test not(evenN)@n

$c_{11} \triangleq$ test $evenN@n$ $\quad$ $c_{12} \triangleq n@ = evenN$

$P_1 \triangleq \{true\} \cup P$ $\quad$ $P_2 \triangleq \{true, even(zero)\}$

$P_3 \triangleq \{true\}$

After a single application of **(rec)**, we can exploit **(relax)** to track only {true, even(0)} as values of n. Then, we stop unfolding the kleene iteration by applying (iterate) and terminate the proof with the final (test) rule. The final post-condition $Q = \{true\}$ satisfies $Spec$, as we expected.

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{P_2}(c_{11})}{[P_2]\ c_{11}\ [\{even(zero)\}]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^A_{\{even(zero)\}}(c_{12})}{[\{even(zero)\}]\ c_{12}[P_3]}\ \text{(assign)}}{\vdash_A\ [P_2]\ c_1\ [P_3]}\ \text{(seq)} \qquad P_3 \subseteq A(P_2) = \{tt, e\}}{(**)}\ \text{(iterate)}$$

$$\dfrac{P_2 \subseteq P_1 \subseteq A(P_2) = \{tt, e\} \qquad \dfrac{(**)}{\vdash_A\ [P_2]\ c_1\ [P_2]}\ \text{(iterate)} \qquad P_2 \subseteq P_2 \subseteq A(P_2) = \{tt, e\}}{(*)}\ \text{(relax)}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{P}(c_{11})}{\vdash_A\ [P]\ c_{11}\ [P]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^A_{P}(c_{12})}{\vdash_A\ [P]\ c_{12}\ [P_1]}\ \text{(assign)}}{\vdash_A\ [P]\ c_1\ [P_1]}\ \text{(seq)} \qquad \dfrac{(*)}{\vdash_A\ [P_1]\ c_1^*\ [P_2]}\ \text{(relax)}}{\vdash_A\ [P]\ c_1^*\ [P_2]}\ \text{(rec)} \qquad \dfrac{\mathbb{C}^A_{P_2}([\![c_2]\!])}{\vdash_A\ [P_2]\ c_2\ [P_3]}\ \text{(test)}}{\vdash_A\ [P]\ c\ [P_3]\}}\ \text{(seq)}$$

Figure 5.5: Derivation of $\vdash_A\ [P = \{even(n) \mid n \in Even\}]\ c\ [\{true\}]$.

$Spec$ is expressible, so we can also reason about the incorrectness of properties for c. Take $P' = P \cup \{even(succ(zero))\}$ instead.

We can obtain the triple $\vdash_A\ [P']\ c\ [\{true, false\}]$ as shown in Figure 5.6, with the following conditions:

$$P'_1 \triangleq \{true, false\} \cup \{even(n) \mid n \in Even\} \qquad P'_2 \triangleq \{true, false, even(zero)\}$$
$$P'_3 \triangleq \{true, false\} \qquad P'_4 \triangleq \{true\}$$

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{P'_2}(c_{11})}{[P'_2]\ c_{11}\ [\{even(zero)\}]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^A_{\{even(zero)\}}(c_{12})}{[\{even(zero)\}]\ c_{12}[P'_4]}\ \text{(assign)}}{\vdash_A\ [P'_2]\ c_1\ [P'_4]}\ \text{(seq)} \qquad P'_4 \subseteq A(P'_2) = \{tt, ff, e\}}{(**)}\ \text{(iterate)}$$

$$\dfrac{P'_2 \subseteq P'_1 \subseteq A(P'_2) = \{tt, ff, e\} \qquad \dfrac{(**)}{\vdash_A\ [P'_2]\ c_1\ [P'_2]}\ \text{(iterate)} \qquad P'_2 \subseteq P'_2 \subseteq A(P'_2) = \{tt, ff, e\}}{(*)}\ \text{(relax)}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{P}(c_{11})}{\vdash_A\ [P']\ c_{11}\ [P']}\ \text{(test)} \qquad \dfrac{\mathbb{C}^A_{P}(c_{12})}{\vdash_A\ [P']\ c_{12}\ [P'_1]}\ \text{(assign)}}{\vdash_A\ [P']\ c_1\ [P'_1]}\ \text{(seq)} \qquad \dfrac{(*)}{\vdash_A\ [P'_1]\ c_1^*\ [P'_2]}\ \text{(relax)}}{\vdash_A\ [P']\ c_1^*\ [P'_2]}\ \text{(rec)} \qquad \dfrac{\mathbb{C}^A_{P'_2}([\![c_2]\!])}{\vdash_A\ [P'_2]\ c_2\ [P'_3]}\ \text{(test)}}{\vdash_A\ [P']\ c\ [P'_3]\}}\ \text{(seq)}$$

Figure 5.6: Derivation of $\vdash_A\ [P']\ c\ [\{true, false\}]$.

$Spec$ is not satisfied, as $\{true, false\} \not\subseteq \{true\}$, and the final value

$false$ is a true alert.

Limitation of the abstract domain

The chosen abstraction A has a very strong limitation: it becomes incomplete quite easily on many input preconditions.

Take $P = \{even(succ^2), even(succ^3)\}$. Any proof involving P will fail the local completeness check for the assignment $c \triangleq$**n@=evenN**, since

$$A[\![c]\!]P = A(\{even(zero), even(succ(zero))\}) = \{e, o\}$$
$$A[\![c]\!]A(P) = A([\![c]\!] \circ \gamma(\{e, o\})) = \{tt, ff, e, o\}$$

It is easy to check that this issue occurs with **any** P such that

$$P \subseteq \{even(succ^n)\ n > 2\}$$

The problem is that, when we first abstract the input P, we add the two values $even(zero)$ and $even(succ(zero))$, which gets rewritten to $true$ and $false$. The chosen abstraction is too coarse for any input not including one of them.

In this situation, the strong point of LCL is clear: by having a local definition of completeness, we can focus only on the computational traces produced by a predefined input P, ignoring any other point for which A may be incomplete. In the previous examples, at least one value between $even(zero)$ and $even(succ(succ(zero)))$ is always included, so we still obtain valid proofs.

Of course, a desirable situation would be to recover completeness, at least for some executions. A possible solution involves locally refining the abstract domain on each incomplete trace. An interesting line of work could be to employ the recent **AIR** methodology [BGGR22], whose purpose is exactly fixing LCL proofs by refining incomplete abstractions.

Chapter 6

Conclusions

We have explored the applicability of Local Completeness Logic (LCL) for strategic rewriting. We have designed a new strategy language, **Strat**, and its store-based semantics. We showed the basic constructs of the language and how to combine them to define more complex programs. Then, we have presented the LCL framework and the extension to verify Strat programs. We have seen the base proof system and the new rule to handle Strat procedures. The extensions were validated by proving that the logic is still sound. Finally, we have seen two examples, where toy programs were analyzed using LCL. We have defined the properties to be verified, and exposed some derivations in the proof system, based on simple non-relational abstract domains.

6.1 Related work

Research on verification of rewriting systems has been ongoing for decades. The typical analysis is **reachability**, where the goal is to approximate the set of reachable by applying rules to a set of initial terms [GT01, FGVTT04]. In these works, a popular approach is to describe set of terms by **tree automata**, a particular form of automata, capable of recognizing tree-like terms. The analysis proceeds with a technique called **tree automata completion** [GR10], where an automaton approximating reachable terms is progressively built.

Abstract interpretation for strategy languages has been recently studied for the **Stratego** language [KE17, KPE18]. The authors propose a new generic approach to build analyzers, introducing a new framework that provides the possibility of defining **compositionally sound** abstract interpreters. The approach has been tested by defining a **tree shape** analysis

that predicts the shape of terms transformed by a Stratego program.

6.2 Future work

A series of possible expansions for this thesis could be investigated:

- **Language features.** A first immediate line of work is enriching **Strat** with more features. In its current version, **Strat** only supports the most basic form of term rewriting, **unsorted** and with unconditional rules. Adding the possibility of defining **sorted values** and handling rules with **conditions** would be a natural extension. Another, more ambitious possibility would be to consider generic terms $t \in T_\Sigma(X)$ as values instead of just ground terms. This would extend the expressivity of the language, but a careful handling of rewrite rules is required: applying a rule $l \to r$ means performing **unification** instead of simple matching for l, and we talk about **narrowing** instead of rewriting. The generalization of Strat to the **narrowing** setting could be of great interest to extend our results.

- **Better abstract domains.** We mentioned in the last chapter how easily an abstract domain for terms can become incomplete. This is an unfortunate consequence of trying to approximate terms of arbitrary depth by stopping at a certain threshold to keep the analysis terminating. Without distinguishing between possible different sub-terms, we frequently end up losing too much information, making proofs in LCL difficult to complete. A first solution was mentioned before: apply some form of refinement to the abstract domain whenever it becomes incomplete. The downside of this approach is that it could make the abstract domain too concrete, impacting the feasibility of the analysis. A more general solution is to adapt already existing abstractions for term rewriting systems to the abstract interpretation context: in particular, we think that exploiting **tree automata techniques**[CDG+08, Tak04, HGJ20] could be a successful strategy to recover completeness in many cases.

- **LCL for functional languages.** LCL is defined as a Hoare-like system, and because of this, it is particularly adapt for the verification of imperative, store-based languages. This is the main reason that motivated us to define Strat as store-based, instead of the standard, expression oriented interpretation of strategy languages. An interesting area

of research would be to apply the same concepts of **local completeness** and **under-approximation** to functional languages. The main issue to confront is recursion: following a functional approach, a strategy iteration $\mathbf{c}^*$ would be represented by the application of a fixpoint operator $\mu\mathbf{x}.\mathbf{c}; \mathbf{x}$. This would probably mean some major changes in the proof system, especially for the rules dealing with iteration. We argue that the definition of a proper verification framework to a functional context could be of great benefit, especially for the application of LCL to already existing strategy languages (**Stratego**, **Maude**, ecc...).

Bibliography

[AMO01] Ravi P Agarwal, Maria Meehan, and Donal O'regan. *Fixed point theory and applications*, volume 141. Cambridge university press, 2001.

[Awo10] Steve Awodey. *Category theory*. Oxford university press, 2010.

[BCDK10] Tony Bourdier, Horatiu Cirstea, Daniel Dougherty, and Hélène Kirchner. Extensional and intensional strategies. *arXiv preprint arXiv:1001.4427*, 2010.

[BGGR21] Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. A logic for locally complete abstract interpretations. In *36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021, Rome, Italy, June 29 - July 2, 2021*, pages 1–13. IEEE, 2021.

[BGGR22] Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. Abstract interpretation repair. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 426–441, 2022.

[BKK+98] Peter Borovanskỳ, Claude Kirchner, Hélene Kirchner, Pierre-Etienne Moreau, and Christophe Ringeissen. An overview of elan. *Electronic Notes in Theoretical Computer Science*, 15:55–70, 1998.

[BM17] Roberto Bruni and Ugo Montanari. *Models of computation*. Springer, 2017.

[BN99] Franz Baader and Tobias Nipkow. *Term rewriting and all that*. Cambridge university press, 1999.

[BP97] Peter Borovanskỳ and Hélène Peter. Strategies of elan: meta-interpretation and partial evaluation. In *Proceedings of the 2nd International Workshop on the Theory and Practice of Algebraic Specifications 2*, pages 1–15, 1997.

[CC77] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '77, page 238–252, New York, NY, USA, 1977. Association for Computing Machinery.

[CC79] Patrick Cousot and Radhia Cousot. Systematic design of program analysis frameworks. In *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '79, page 269–282, New York, NY, USA, 1979. Association for Computing Machinery.

[CC92] Patrick Cousot and Radhia Cousot. Abstract interpretation frameworks. *Journal of logic and computation*, 2(4):511–547, 1992.

[CDD+15] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O'Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. Moving fast with software verification. In *NASA Formal Methods Symposium*, pages 3–11. Springer, 2015.

[CDE+20] Manuel Clavel, Francisco Durán, Steven Eker, Santiago Escobar, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, Rubén Rubio, and Carolyn Talcott. Maude manual (version 3.0). *SRI International*, 2020.

[CDG+08] Hubert Comon, Max Dauchet, Rémi Gilleron, Florent Jacquemard, Denis Lugiez, Christof Löding, Sophie Tison, and Marc Tommasi. Tree automata techniques and applications, 2008.

[Coh12] Paul Moritz Cohn. *Universal algebra*, volume 6. Springer Science & Business Media, 2012.

[Cou21] Patrick Cousot. *Principles of Abstract Interpretation.* MIT Press, 2021.

[DFLO19] Dino Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W O'Hearn. Scaling static analyses at facebook. *Communications of the ACM*, 62(8):62–70, 2019.

[DP02] Brian A Davey and Hilary A Priestley. *Introduction to lattices and order*. Cambridge university press, 2002.

[FGVTT04] Guillaume Feuillade, Thomas Genet, and Valérie Viet Triem Tong. Reachability analysis over term rewriting systems. *Journal of Automated Reasoning*, 33(3):341–383, 2004.

[GD03] Andrzej Granas and James Dugundji. *Fixed point theory*, volume 14. Springer, 2003.

[GLR15] Roberto Giacobazzi, Francesco Logozzo, and Francesco Ranzato. Analyzing program analyses. *ACM SIGPLAN Notices*, 50(1):261–273, 2015.

[GR10] Thomas Genet and Vlad Rusu. Equational approximations for tree automata completion. *Journal of Symbolic Computation*, 45(5):574–597, 2010.

[Grä08] George Grätzer. *Universal algebra*. Springer Science & Business Media, 2008.

[GT01] Thomas Genet and Valérie Viet Triem Tong. Reachability analysis of term rewriting systems with timbuk. In *International Conference on Logic for Programming Artificial Intelligence and Reasoning*, pages 695–706. Springer, 2001.

[HGJ20] Timothée Haudebourg, Thomas Genet, and Thomas Jensen. Regular language type inference with term rewriting. *Proceedings of the ACM on Programming Languages*, 4(ICFP):1–29, 2020.

[HMU07] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to automata theory, languages, and computation, 3rd Edition*. Pearson international edition. Addison-Wesley, 2007.

[Hoa69] Charles Antony Richard Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.

[KE17] Sven Keidel and Sebastian Erdweg. Toward abstract interpretation of program transformations. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Meta-Programming Techniques and Reflection*, pages 1–5, 2017.

[KK90] Jan Willem Klop and JW Klop. *Term rewriting systems*. Centrum voor Wiskunde en Informatica, 1990.

[Kle52] Stephen Cole Kleene. Introduction to metamathematics. 1952.

[KPE18] Sven Keidel, Casper Bach Poulsen, and Sebastian Erdweg. Compositional soundness proofs of abstract interpreters. *Proceedings of the ACM on Programming Languages*, 2(ICFP):1–26, 2018.

[LRV+22] Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W O'Hearn. Finding real bugs in big programs with incorrectness logic. *Proceedings of the ACM on Programming Languages*, 6(OOPSLA1):1–27, 2022.

[MYE21] Toby Murray, Pengbo Yan, and Gidon Ernst. Incremental vulnerability detection with insecurity separation logic. *arXiv preprint arXiv:2107.05225*, 2021.

[NNH04] Flemming Nielson, Hanne R Nielson, and Chris Hankin. *Principles of program analysis*. Springer Science & Business Media, 2004.

[O'H19] Peter W O'Hearn. Incorrectness logic. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–32, 2019.

[Ohl02] Enno Ohlebusch. *Advanced topics in term rewriting*. Springer Science & Business Media, 2002.

[RMOPV18] Rubén Rubio, Narciso Martí-Oliet, Isabel Pita, and Alberto Verdejo. Parameterized strategies specification in maude. In *International Workshop on Algebraic Development Techniques*, pages 27–44. Springer, 2018.

[RMOPV22] Rubén Rubio, Narciso Martí-Oliet, Isabel Pita, and Alberto Verdejo. Model checking strategy-controlled systems in rewriting logic. *Automated Software Engineering*, 29(1):1–62, 2022.

[Ros95] Mads Rosendahl. Introduction to abstract interpretation. *Computer Science University of Copenhagen*, 1995.

[Tak04] Toshinori Takai. A verification technique using term rewriting systems and abstract interpretation. In *International Conference on Rewriting Techniques and Applications*, pages 119–133. Springer, 2004.

[Tar55] Alfred Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific journal of Mathematics*, 5(2):285–309, 1955.

[V+98] Eelco Visser et al. A core language for rewriting. *Electronic Notes in Theoretical Computer Science*, 15:422–441, 1998.

[Vis01] Eelco Visser. Stratego: A language for program transformation based on rewriting strategies system description of stratego 0.5. In *International Conference on Rewriting Techniques and Applications*, pages 357–361. Springer, 2001.

[Vis04] Eelco Visser. Program transformation with stratego/xt. In *Domain-specific program generation*, pages 216–238. Springer, 2004.

[Win93] Glynn Winskel. *The formal semantics of programming languages: an introduction.* MIT press, 1993.

Appendix A

Partially Ordered Sets

This chapter provides a brief introduction to the theory of *partial orders.*. A basic knowledge of set theory is assumed. For a more in depth presentation of order theory, the reader can consult one of the many books that explore the subject [DP02, NNH04, BM17].

Function notation Given a set X, the identity function mapping an element of X to itself is denoted id_X, and the subscript is omitted when the context is clear. If $f : X \to Y$, f is overloaded to denote its (collecting) lifting to powersets $f : \mathcal{P}(X) \to \mathcal{P}(Y)$:

$$f(S) = \{f(x) \mid x \in S\}$$

Given $f : X \to Y$ and $g : Y \to Z$, their composition is denoted as $g \circ f$, or simply gf. For $f : X \to X$ and $n \in \mathbb{N}$ we let $f^n : X \to X$ be defined inductively as: $f^0 \triangleq id_X$ and $f^{n+1} \triangleq f \circ f^n$.

Definition A.1 (Partial order)**.** Given a set S, a *partial order* is a binary relation $\sqsubseteq_S \subseteq S \times S$ which is

$$\begin{aligned} \textbf{Reflexive:} \quad & \forall s \in S, s \sqsubseteq_S s \\ \textbf{Antisymmetric:} \quad & \forall s, t \in S, s \sqsubseteq_S t \wedge t \sqsubseteq_S s \implies s = t \\ \textbf{Transitive:} \quad & \forall s, t, u \in S, s \sqsubseteq_S t \wedge t \sqsubseteq_S u \implies s \sqsubseteq_S u \end{aligned}$$

We call $(S, \sqsubseteq_S)$ a *poset* (for partially ordered set). When no confusion can arise, we will omit the subscript S from $\sqsubseteq_S$.

A poset $(S, \sqsubseteq)$ such that $\forall s, t \in S. s \sqsubseteq t \vee t \sqsubseteq s$ is called a **total order**.

Definition A.2 (Dual order)**.** Given a poset $(S, \sqsubseteq)$, the **dual poset** is the poset $(S, \sqsubseteq_S^{-1})$ where a $\sqsubseteq^{-1}$ b if and only if b $\sqsubseteq$ a .

We will use $\sqsupseteq$ to denote the dual order.

Hasse diagrams It is useful to have a graphical representation of posets. The notation that we will use is Hasse diagrams. Given a poset $(S, \sqsubseteq)$, its Hasse diagram is a graph where nodes are elements $s \in S$ and there is an edge between a and b if and only if $a \sqsubseteq b$. The fundamental property of a Hasse diagram is that it omits all the information that can be implicitly derived by the properties of a poset. In particular, all the edges that can be inferred by the reflexive and transitive properties are omitted. An example of this representation is given in Figure A.1, where a diagram for the poset defined by a powerset ordered by inclusion is shown.

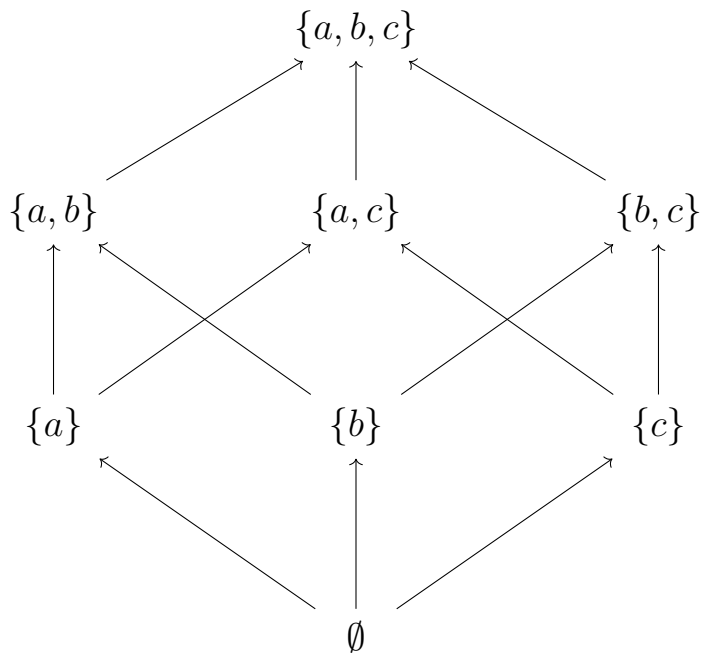


Figure A.1: The Hasse diagram for $(\mathcal{P}(\{a, b, c\}), \subseteq)$.

Definition A.3. Let $(S, \sqsubseteq)$ be a poset and let $M \subseteq A$.

- $m \in M$ is a **maximal** element of M if $\forall n \in M. m \sqsubseteq n \implies n = m$.
- $m \in M$ is the **greatest** element (maximum) of M if $\forall n \in M. n \sqsubseteq m$.
- $u \in A$ is an **upper bound** of M if $\forall m \in M. m \sqsubseteq u$.

Definition A.4 (Least upper bound)**.** Given a poset $(S, \sqsubseteq)$ and a subset $A \subseteq S$, $s \in S$ is the **least upper bound (lub)** of A if and only if s is the least element of the upper bounds of A. Formally:

1. s is an upper bound: $\forall a \in A, a \sqsubseteq s$.

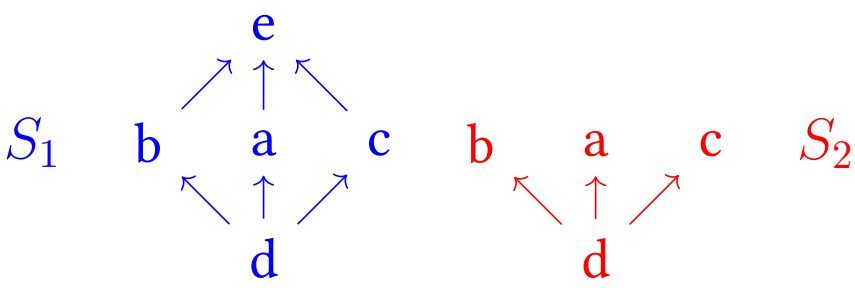


Figure A.2: A visualization of the difference between **maximum** and **maximal** element. S_1 has greatest element e, while S_2 has a, b, and c as maximal elements but no maximum (because they are not comparable).

2. for any upper bound u of A, then $s \sqsubseteq u$ $(\forall u \in S.(\forall a \in A.a \sqsubseteq u) \implies s \sqsubseteq u)$.

If we consider the dual order $(S, \sqsupseteq)$, we obtain the notions of **minimal** element, **least** element (minimum), **lower bound** and **greatest lower bound (glb)**. We denote lubs with the notation $\bigsqcup A$ and glbs with the notation $\bigsqcap A$.

Top and Bottom As extreme cases, we can take the set S itself or $\emptyset$ as the subset A:

- For S, we will call $\bigsqcup S$ the **top** element and write $\top$.
- For $\emptyset$, we will call $\bigsqcap \emptyset$ the **bottom** element, and write $\bot$.

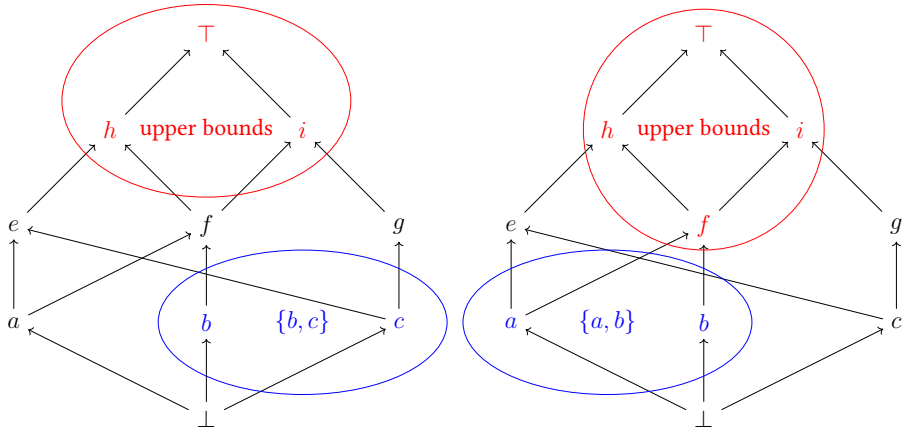
Note that, in general, a set S is not required to have a lub or a glb, but when it does, it is unique. When we have $S = \{a, b\}$, we will use $a \sqcup b$ $(a \sqcap b)$ instead [1]

Example A.5. (lub) Figure A.3 shows the concept of **least upper bound**. The subset $\{b, c\}$ in Figure A.3a has $\{h, i, \top\}$ as its set of upper bounds. This set has no least element (h and i are not comparable), so {b, c} has no lub. Figure A.3b shows that the set of upper bounds of $\{a, b\}$ is $\{f, h, i, \top\}$. In this case, f is the least element and thus the lub of $\{a, b\}$.

Definition A.6. (Chains) Let $(S, \sqsubseteq)$ be a poset, a function $C : \mathbb{N} \to S$ is called an *ascending chain* if:

$$\forall n \in \mathbb{N}.C(n) \sqsubseteq C(n+1)$$

[1] In the literature, a least upper bound is also called **join**, while greatest lower bounds are called **meets**.



(a) Subset$\{b, c\}$ has no lub. (b) Subset$\{a, b\}$ has lub f.

Figure A.3: Two subsets of a poset and their upper bounds.

We denote chains by $C = \{d_i\}_{i \in \mathbb{N}}$, where $\forall i \in \mathbb{N}.\ d_i = C(i)$, i.e.,

$$d_0 \sqsubseteq d_1 \sqsubseteq d_2 \ldots$$

A chain C is *finite* if its image is a finite set. Otherwise, we say that C is *infinite*.

Note that, by reflexivity, a finite chain still has infinite elements, but only finitely many different ones. In particular, there is an index k and one element d such that $\forall i \geq k.d_i = d$.

A poset satisfies the **ascending chain condition (ACC)** if and only if each ascending chain is finite.

By taking the dual order, we obtain **descending chains** and the **descending chain condition (DCC)**.

Definition A.7. (Limit) Let C be a chain. The lub of the image of C, when it exists, is called the *limit* of C.

If $C = \{d_i\}_{i \in \mathbb{N}}$ we write $d = \bigsqcup_{i \in \mathbb{N}} d_i$.

Definition A.8. (Complete partial order) A poset $(S, \sqsubseteq)$ is called **complete** (CPO) if each chain has a limit.

If S has least element $\perp$, we call it a *CPO with bottom (CPO$_\perp$)*.

Definition A.9. (Lattice and complete lattice)

Let S be a non-empty ordered set.

- If $x \sqcup y$ and $x \sqcap y$ exist for all $x, y \in S$, S is called a **lattice**.
- If $\bigsqcup A$ and $\bigsqcap A$ exist for all $A \subseteq S$, then S is called a **complete lattice**.

It is easy to see that the poset from Figure A.1 is a complete lattice. Actually, given *any* set X, its powerset $\mathcal{P}(X)$ forms a complete lattice $(\mathcal{P}(X), \subseteq, \cup, \cap, \emptyset, X)$.

Definition A.10. (Monotone function) Given two posets S and T, a function $f : S \to T$ is called **monotone** if

$$\forall s, t \in S.(s \sqsubseteq_S t) \implies f(s) \sqsubseteq_T f(t).$$

In other words, a *monotone function* preserves the order of elements.

Example A.11. (Non-monotone function). Let us consider the poset $(\mathbb{N} \cup \{\infty\}, \leq)$ and the function $f(n) = n\%2$ (and $f(\infty) = \infty$) that computes the remainder of $n \div 2$. This function is **not** monotone, indeed $1 \leq 2$ but $f(1) = 1 \not\leq f(2) = 0$ (see Figure A.4).

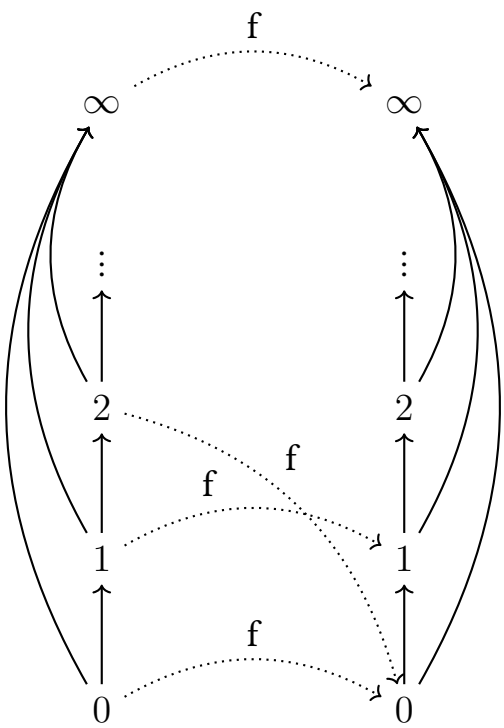


Figure A.4: A non-monotone function.

Given a set S and a poset T, the set of functions from S to T can also be extended to a poset.

Definition A.12. (Order of functions) Let $f, g : S \to T$ be two functions from set S to poset T, we say that $f \sqsubseteq g$ if

$$\forall s \in S.f(s) \sqsubseteq_T g(s).$$

Moreover, if T is a (complete) lattice, the set of functions from S to T is a (complete) lattice too. This holds even if we restrict S to be a poset and the functions from S to T to be monotone.

Definition A.13. (Additive function) Given two posets S and T, a function $f : S \to T$ is called **additive** (or join morphism) if

$$\forall s, t \in S. f(s \sqcup t) = f(s) \sqcup f(t).$$

We say that f *preserves arbitrary lubs*. If we take $s \sqcap t$ instead, we obtain a **co-additive function (meet morphism)**, and say that f *preserves arbitrary glbs*.

Definition A.14. (Continuity). Let $(D, \sqsubseteq_D)$ and $(E, \sqsubseteq_E)$ be two CPOs and let $f : D \to E$ be a monotone function. We say that f is **continuous** if, for each chain in D, we have:

$$f(\bigsqcup_{i \in \mathbb{N}} d_i) = \bigsqcup_{i \in \mathbb{N}} f(d_i)$$

Informally, this means that f *preserves limits*. Notice that the limit in the left-hand side is taken in D, while the one on the right is taken in E.

Continuity is closed under composition, meaning that composing two continuous functions still yields a continuous function.

A.1 Fixpoints

We are now ready to see the concept of **fixpoints**.

Definition A.15. (Pre-fixpoint and fixpoint) Let $f : P \to P$ be a monotone function on a poset $(P, \sqsubseteq)$. An element p is a **pre-fixpoint** if

$$f(p) \sqsubseteq p$$

An element $d \in D$ is a **fixpoint** of f if

$$f(d) = d$$

It is trivial to see that every fixpoint is also a pre-fixpoint (by reflexivity of $\sqsubseteq$). We will indicate with **gfp(**f**)** the greatest fixpoint of f, and with **lfp(**f**)** the least fixpoint of f, when they exist.

Dually, we can define a **post-fixpoint** of f as an element p such that $p \sqsubseteq f(p)$.

There are many important theorems on the existence of fixpoints [AMO01, GD03]. For our purposes, we are interested in two main results:

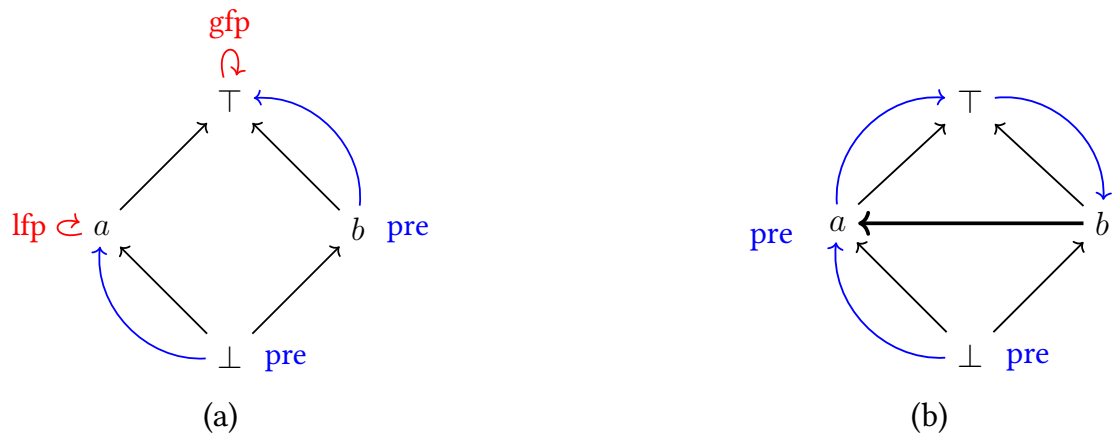


Figure A.5: Functions on posets (red and blue arrows). (a) is monotone and has two fixpoints, namely a and $\top$. (b) is not monotone and has no fixpoint.

- Knaster-Tarski Theorem [Tar55].
- Kleene fixpoint theorem [Kle52].

Theorem A.16. *(Knaster-Tarski theorem) Let $f : L \to L$ be a monotone function on a* ***complete lattice*** *L. Then, the set of all fixpoints of f is also a complete lattice $(P, \sqsubseteq)$ with:*

- $\mathit{gfp}(f) = \bigsqcup P = \bigsqcup\{x \in L \mid x \sqsubseteq f(x)\}$
- $\mathit{lfp}(f) = \bigsqcap P = \bigsqcap\{x \in L \mid f(x) \sqsubseteq x\}$

This theorem guarantees us that, for complete lattices, we can always find the greatest and least fixpoint. More specifically, the greatest fixpoint is defined as the leasìt upper bound of the set of post-fixpoints, while the least fixpoint is the greatest lower bound of the set of pre-fixpoints.

Knaster-Tarski theorem is fundamental, not only for the existence of the least (greatest) fixpoint p, but also for its characterization as the glb (lub) of the set of pre-fixpoints. In the context of program verification, it means than **any** pre-fixpoint of f is a sound approximation of the lfp p.

Theorem A.17. *(Kleene fixpoint theorem) Let $f : D \to D$ be a continuous function on a CPO$_\perp$ D. Then, let*

$$\mathit{fix}(f) = \bigsqcup_{n \in \mathbb{N}} f^n(\perp)$$

The element $\mathit{fix}(f) \in D$ has the following properties:

1. *$\mathit{fix}(f)$ is a fixpoint of f*

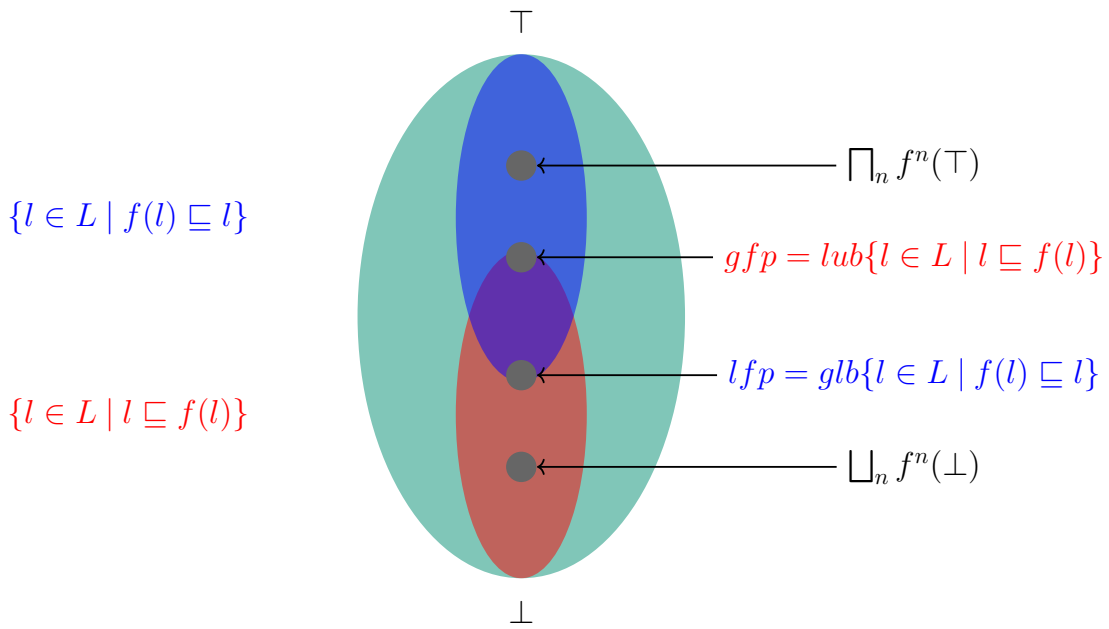


Figure A.6: Knaster-Tarski theorem.

2. *fix*(f) *is the least pre-fixpoint of* f*, namely*

$$f(d) \sqsubseteq d \implies \mathit{fix}(f) \sqsubseteq d$$

Since any fixpoint is also a pre-fixpoint, fix(f) *is also the least fixpoint of* f*.*

Kleene theorem requires stronger hypotheses than Theorem A.16 on the function f, i.e., continuity instead of monotonicity, but it also requires only a CPO$_\bot$ instead of a complete lattice. More interestingly, it is a *constructive theorem*: it expresses the lfp as the limit of an iteration, by computing the chain $\{f^i(\bot)\}_{i \in \mathbb{N}}$. One could see it as a basic algorithm for computing the least fixpoint.

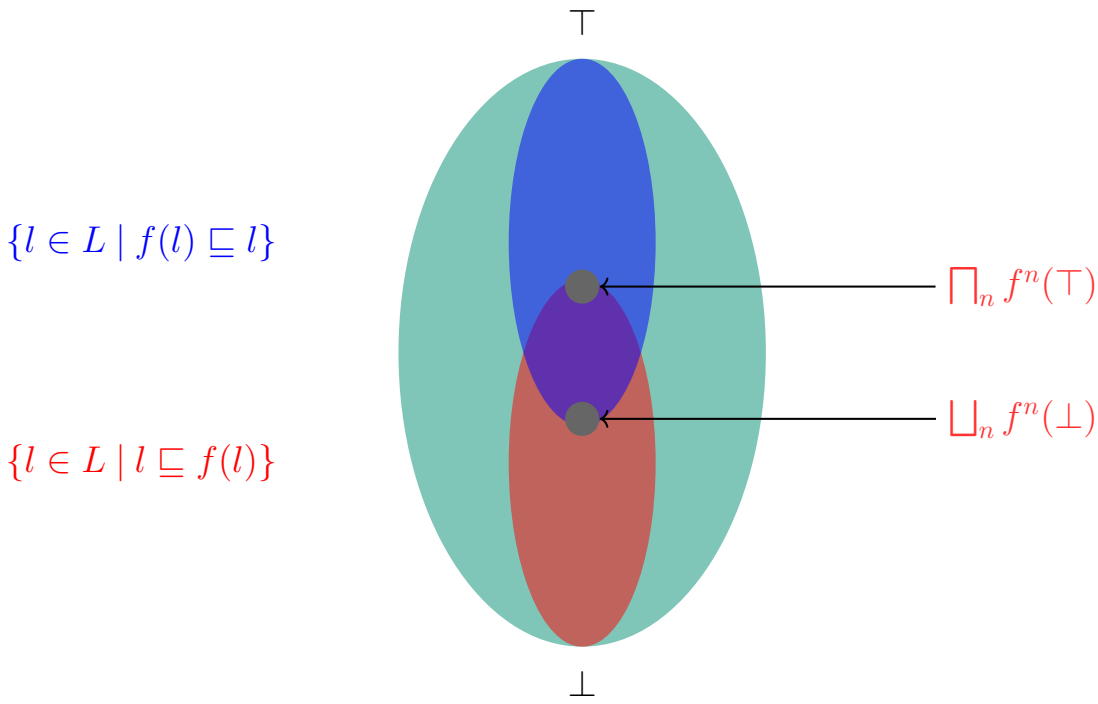


Figure A.7: Kleene fixpoint theorem.